

[두산로보틱스] 지능형 로봇틱스 엔지니어

DOOSIM

환자 침대 자율 이송 시스템

협동3 : 디지털 트윈 기반 로봇 자동화 시뮬레이션 시스템 구현 프로젝트

C-3 | 김한준 · 강대환 · 서정민 · 류호현 · 조해벽

[멘토] 손미란 강사님



DIGITAL TWIN
HOSPITAL

K-Digital Training

목 차

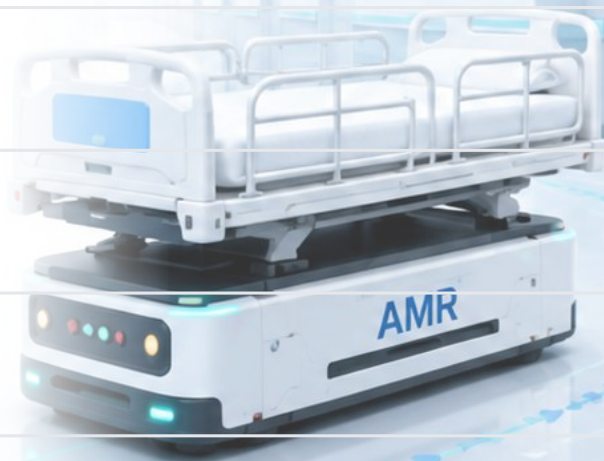
01 프로젝트 개요

02 프로젝트 팀 구성 및 역할

03 프로젝트 수행 절차 및 방법

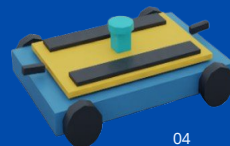
04 프로젝트 수행 경과

05 자체 평가 의견



01 프로젝트 개요

1. 프로젝트 주제 및 선정 배경, 기획 의도
2. 프로젝트 내용
3. 활용 장비 및 재료
4. 프로젝트 구조
5. 활용방안 및 기대 효과



01 프로젝트 개요

1. 프로젝트 주제 및 선정 배경, 기획 의도

병원 침대 이송 시스템 - 디지털 트윈 기반 사전 검증

Isaac Sim 기반 디지털 트윈 환경에서 AMR이 병상에 자동 결합하고, 환자 정보를 확인한 뒤 Nav2를 연계하여 목적지까지 병상을 이송하는 전 과정을 사전 검증하는 프로젝트

1 주제 및 선정 배경

병원 내 반복적 침대 이송 업무를 AMR과 디지털 트윈으로 자동화

2 특화 포인트

OCR 환자 확인, 자기식 결합, 복층 Nav2, 자동 엘리베이터 전환

3 프로젝트 내용

환자 검증 → 침대 하부 진입 → 결합 → MRI 이송 → 복귀 흐름 구현

4 활용 장비·재료

Isaac Sim, ROS 2 Humble, Nav2, LiDAR, RGB-Depth Camera

5 기대 효과

반복 이송 부담 감소, 안전 경로 검증, 병원 자동화 적용 가능성 확인

01 프로젝트 개요

1. 프로젝트 주제 및 선정 배경, 기획 의도

현장 문제

병실·검사실·MRI실 이동은
반복적이지만 침대 자체의 크기와
무게로 인해 의료진의 부담이 크다.

자동화 필요성

환자 이동 경로가 정형화된 경우
AMR이 이송을 보조하면 의료진은
환자 상태 관찰과 핵심 업무에 집중할
수 있다.

DOOSIM 선정

Isaac Sim 병원 환경에서
AMR·침대·엘리베이터·센서를 통합해
실제 도입 전 구현 가능성을 검증한다.



선정 결론 | “환자를 다른 이동 장비로 옮기지 않고 침대 자체를 **AMR**로 이송하는 디지털 트윈 기반 병원 자동화 시스템”으로 주제 선정

01 프로젝트 개요

1. 프로젝트 주제 및 선정 배경, 기획 의도

대형병원 몰린 간호사, 지방은 텅 비었다... 지역차 최대 140배

이유주 기자 | 송인 2026.05.06 11:01 | 댓글 0



| 인구 1000명당 간호사 부산 서구 47.11명... 인제-고성군위는 1명도 안 돼

[메이비뉴스 이유주 기자]



대형병원 중심의 인력 집중 현상으로 지역별 간호사 밀도 격차가 최대 140배에 달하는 것으로 나타났다. ©메이비뉴스

대형병원 중심의 인력 집중 현상으로 지역별 간호사 밀도 격차가 최대 140배에 달하는 것으로 나타났다.

대한간호협회가 4월 건강보험심사평가원의 '전국 간호사 현황(2025)' 자료를 분석한 결과, 지난해 말 기준 간호사 면허 소지자는 약 55만 명에 이르렀다. 그러나 이 가운데 요양기관에서 실제 근무 중인 간호사는 29만 8554명으로, 전체의 54% 수준에 머물렀다. 인구 1000명당 활동 간호사 수는 평균 5.84명으로 집계됐다.

최신뉴스

"지방 간호사 노동강도 '서울의 10배'"...인력 양극화 극심

송고 2026-05-12 11:43

세 줄 요약

구글에서 선호하는 출처로 추가



고유선 기자
+ 구독

대한간호협회 "간호사 육체적·정신적 소진, 환자 안전과 직결"

(서울=연합뉴스) 고유선 기자 = 수도권 대형병원과 지방 중소병원의 인력 격차가 심화하면서 지역 일부 의료기관에서는 간호사 1명이 서울 대형병원보다 최대 10배 수준의 환자 부담을 지고 있다는 지적이 나왔다.

01 프로젝트 개요

병원 침대 이송의 핵심 문제

1 반복 이송 업무 부담

병실·검사실·MRI실 사이의 이동이 반복적으로 발생하며, 침대 이동에 의료진 시간이 지속적으로 사용된다.

2 긴 침대의 충돌·사각지대

침대는 길고 회전 반경이 커서 좁은 복도와 코너에서 벽·사람·장애물과의 충돌 위험이 높다.

3 환자·침대 식별 오류 가능성

여러 환자와 병상이 동시에 존재할 때, 잘못된 침대나 환자를 목적지로 이송할 위험이 있다.

4 층간 이동과 시스템 연동 복잡성

엘리베이터 탑승, 1층·2층 지도 전환, 검사실 도착 확인 등 여러 시스템이 순서대로 맞물려야 한다.

핵심 문제 | 병원 침대 이송은 “이동”만의 문제가 아니라, 큰 병상을 안전하게 식별·결합·주행·층간 이동시켜야 하는 통합 자동화 문제이다.

01 프로젝트 개요

2. 프로젝트 내용

주요 기능

- 환자/침대 식별
- 복도 중앙 자율주행
- 엘리베이터 연동 층간 이동
- 실시간 상태 모니터링
- 충돌 회피 및 안전 주행

프로젝트 컨셉

- 특정 도메인 시스템 표준화 구축
병원 이송 시나리오를 절차화·모델화
- 완전 무인화 지향
반복 이송 업무의 자동화
- 정책 제언 가능
운영 기준·안전 절차 확장 가능

ROKEY 훈련 기반 주요 기능 구현

학습 내용과 프로젝트 적용 연계

Python

OpenCV / AI Vision

ROS2 통신

Nav2 자율주행

Isaac Sim

1 환자 정보 인식



- 카메라 영상 기반 OCR 환자 확인
- 이름 / 생년월일 매칭
- 대상 환자 자동 판별

ROKEY 연계: OpenCV · AI Vision

2 자율주행 및 복도 중앙 주행



- Nav2 기반 경로 계획
- 복도 중앙 유지 주행
- 안전한 병상 운송 경로 수행

ROKEY 연계: ROS2 · Nav2

3 병상 결합 및 이송 시나리오



- 병상 결합 / 해제 제어
- 환자실 → 엘리베이터 → 목적지 이송
- 미션 상태 기반 시나리오 관리

ROKEY 연계: Python · 시스템 제어

4 층간 이동 및 시스템 통합



- 엘리베이터 연동 제어
- 맵 전환 및 층간 이동
- ROS2 노드 간 통신 통합

ROKEY 연계: ROS2 · Isaac Sim



ROKEY에서 학습한 **Python**, **Vision**, **ROS2**, **자율주행** 기술을 실제 병원 **AMR** 프로젝트에 적용하여
환자 인식부터 병상 이송, 층간 이동까지 통합 시스템으로 구현

01 프로젝트 개요

3. 활용 장비 및 내용

기술 스택



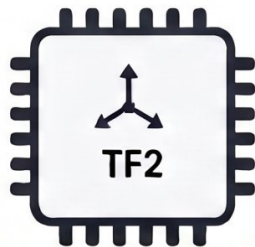
ISAAC SIM 5.1.0
시뮬레이션 환경



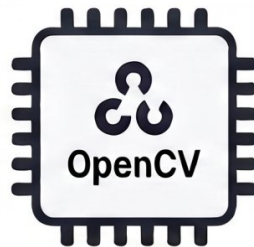
핵심 OS 환경
(Python, C++ 기반)



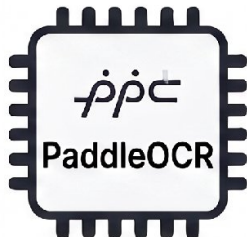
AMR 자율 주행 및
경로 계획 (SLAM 등)



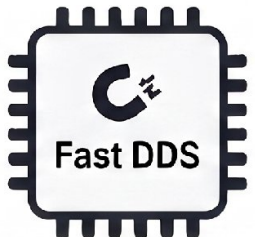
좌표 변환 및
상대 위치 계산



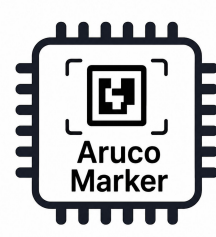
카메라 기반
비전 인식 및 전처리



환자 명찰 및 정보 인식
(OCR 판별)



고속 데이터
통신 백엔드



아루코 마커 기반
위치 인식 및 정렬
(도킹 / 좌표 보정)

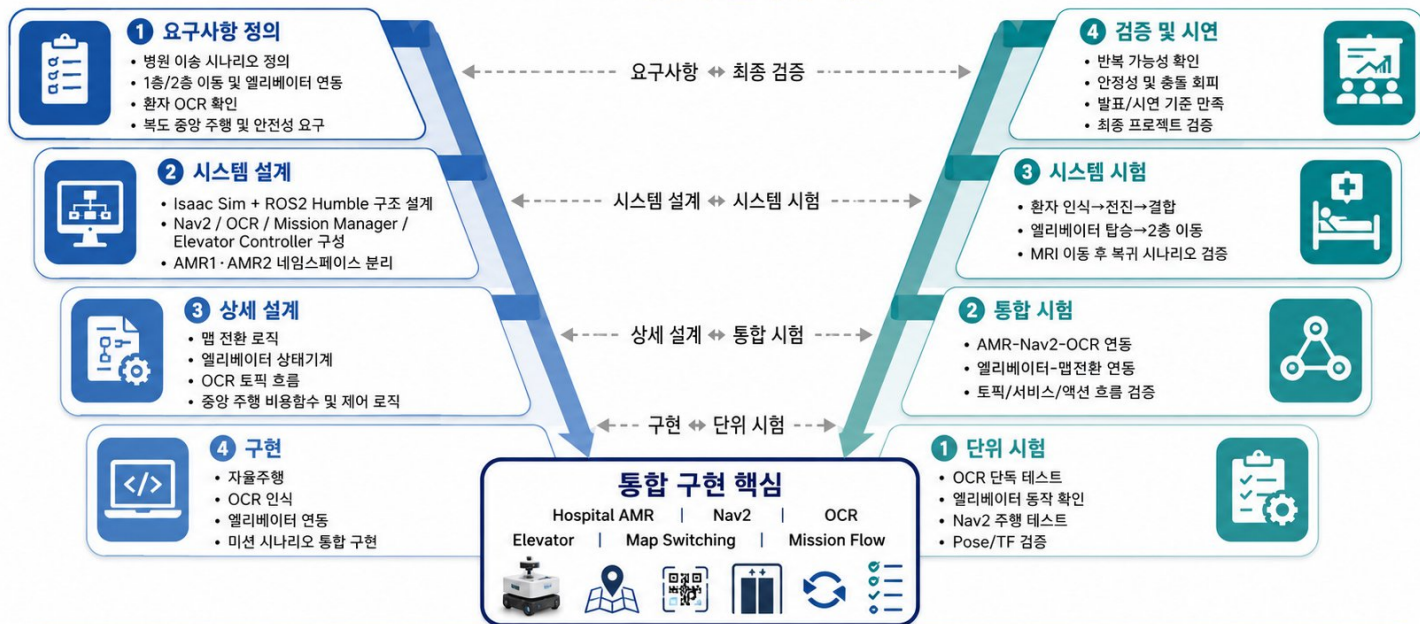
01 프로젝트 개요

4. 프로젝트 구조

병원 AMR 프로젝트 개발 프로세스

V-Model 기반의 절차적 개발

주먹구구식 개발이 아닌
요구사항-설계-구현-검증의 체계적 수행



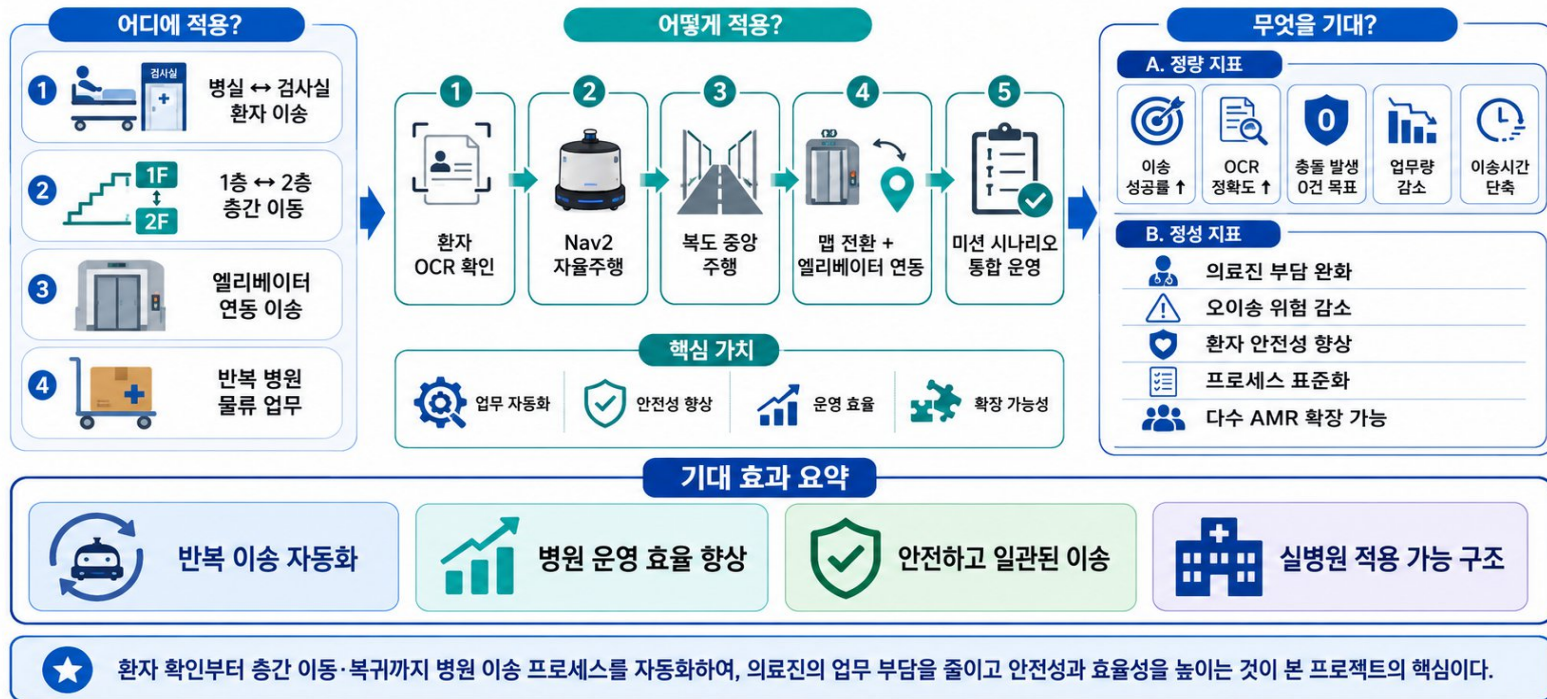
핵심 메시지: 본 프로젝트는 요구사항 분석부터 설계, 구현, 시험, 최종 검증까지 단계적으로 수행한 절차적 개발 프로젝트이다.

01 프로젝트 개요

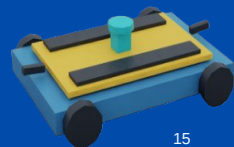
5. 활용 방안 및 기대 효과

활용 방안 및 기대 효과

Business Requirement 가치의 실제 적용과 기대 성과



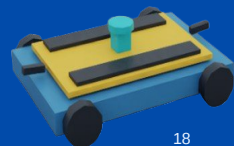
02 프로젝트 팀 구성 및 역할



02 프로젝트 개요

팀원	역할	담당 업무
김한준	팀원	map 2층 담당 , 협동운송 , 협동회피 , 기능 단위별 TEST
강대환	팀원	자율주행, 충돌회피 , map 1층 담당, 기능 단위별 TEST
서정민	팀장	amr 에셋 개발 , 영상제작 , 문서제작 , 노드별 통합, 협동운송, ROS통신 설계
류호현	팀원	GUI, 최종통합 , 기본 Asset searching , 통합 TEST
조해벽	팀원	기본 Asset TEST , 충돌회피 , 기능 단위별 TEST , 최종 기능별 디버깅 작업
손미란	멘토	주제 선정 피드백, 프로젝트 질의 응답

03 프로젝트 수행 절차 및 방법



03 프로젝트 수행 절차 및 방법

구분	기간	활동	비고
사전 기획	07/30	프로젝트 기획 및 주제 선정	아이디어 선정
현장 검증 및 기술 검증	07/31	필수 기능 요구사항 정의 및 설계	
단위 개발 및 테스트	08/01 ~ 08/06	구체적인 설계 및 기능 추가	
통합 시스템 구축 및 테스트	08/07 ~ 08/10	전체 시스템 통합	
시연 및 발표	08/12	팀별 최종 프로젝트 시연	
총 개발기간	07/30 ~ 08/11 (총 13일)		

전체 시나리오 시연 영상

FULL SCENARIO DEMO VIDEO



04 프로젝트 수행 경과

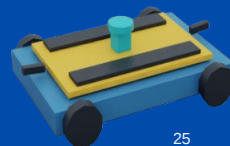
System/Node Architecture & (Function)Flowchart

GUI

ASSET

알고리즘

KEY ISSUE



04 프로젝트 수행 경과

System Architecture

두심이 병원

디지털 트윈 기반 AMR 환자 이송 시스템

2대의 AMR과 2대의 고성능 PC를 기반으로 환자 이송, OCR 확인, 자율주행, 엘리베이터 연동, 실시간 모니터링을 통한 운영



04 프로젝트 수행 경과

Flowchart

병원 AMR 프로젝트 전체 동작 순서

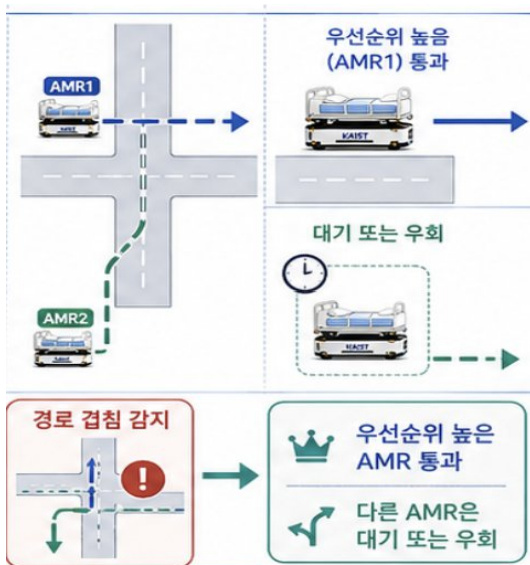


04 프로젝트 수행 경과

플로우차트 속 협동 운영 Mission 2

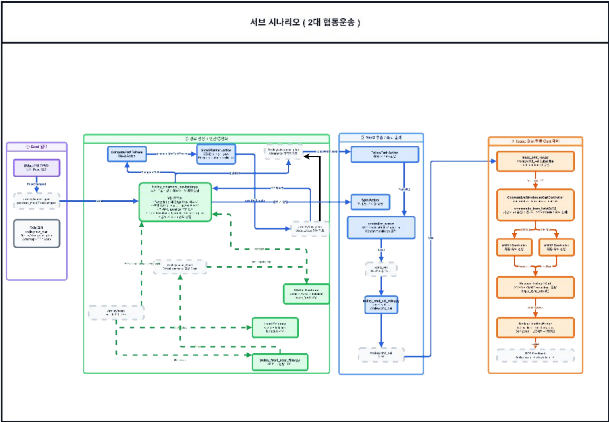
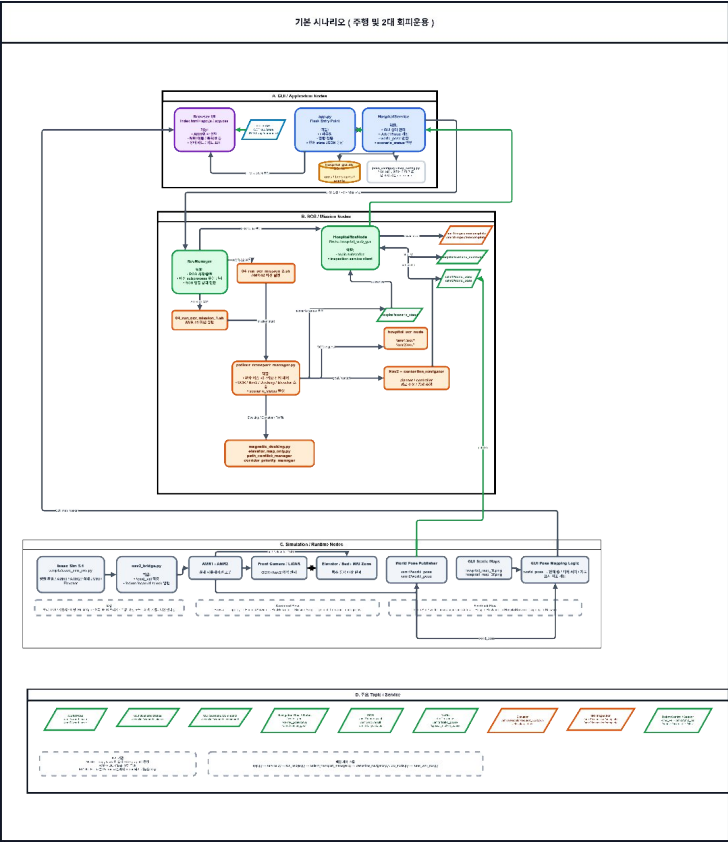


1. 충돌 회피 / 2. 협동 운송



04 프로젝트 수행 경과

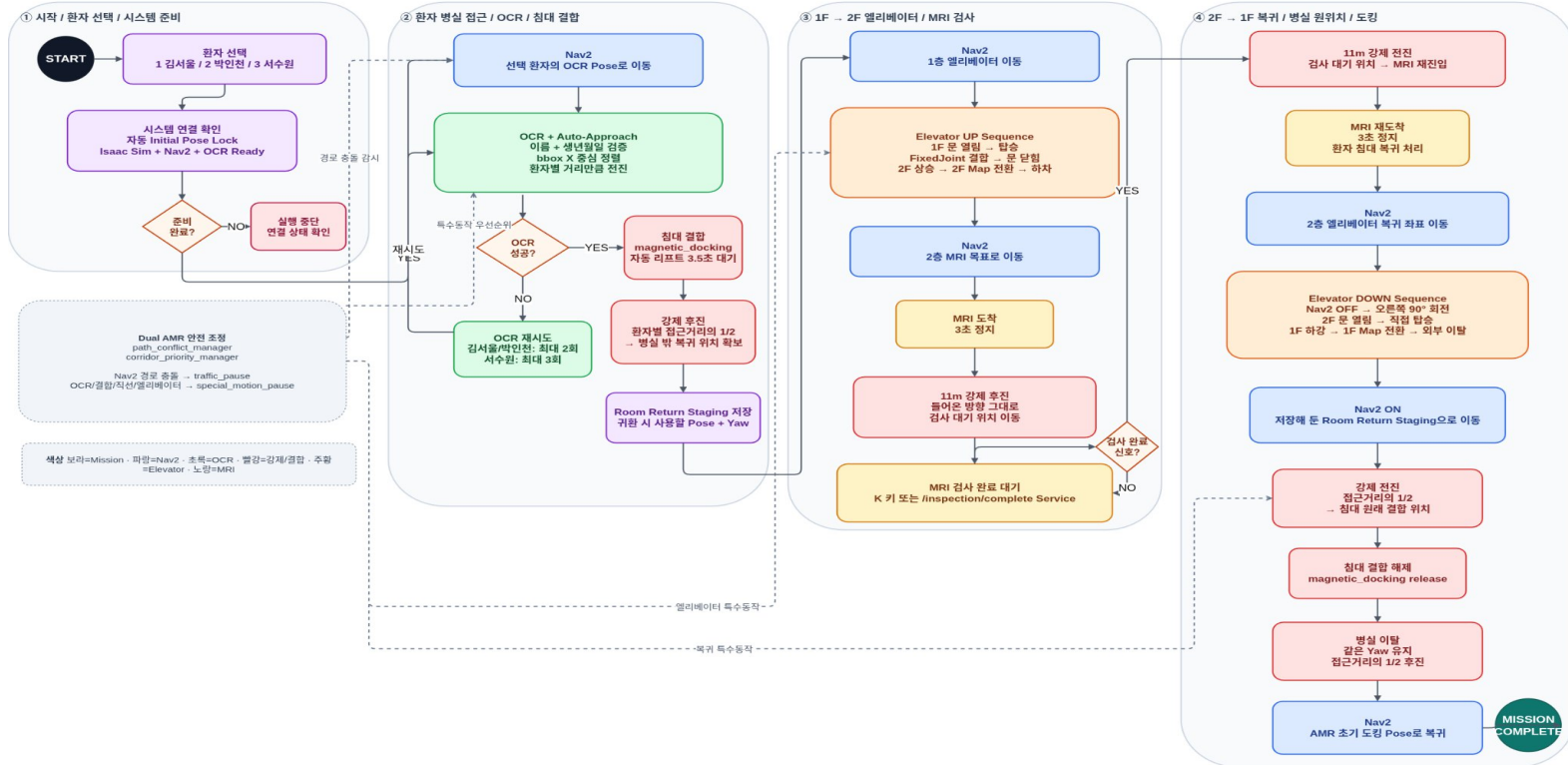
System Architecture



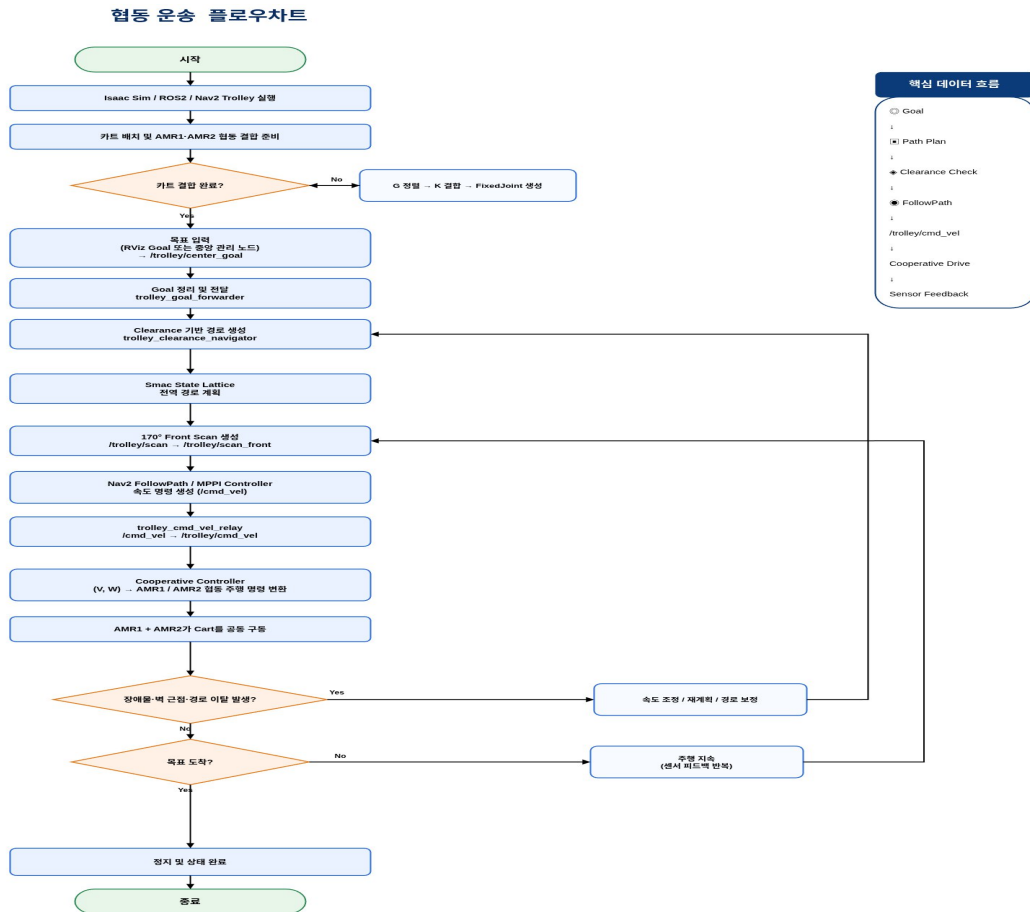
04 MAIN 시나리오 Flowchart

병원 AMR 최종 시나리오 Flowchart

Patient Transport Manager 기준 - AMR1/AMR2 공통 - ROS_DOMAIN_ID = 115



04 Sub 시나리오 Flowchart

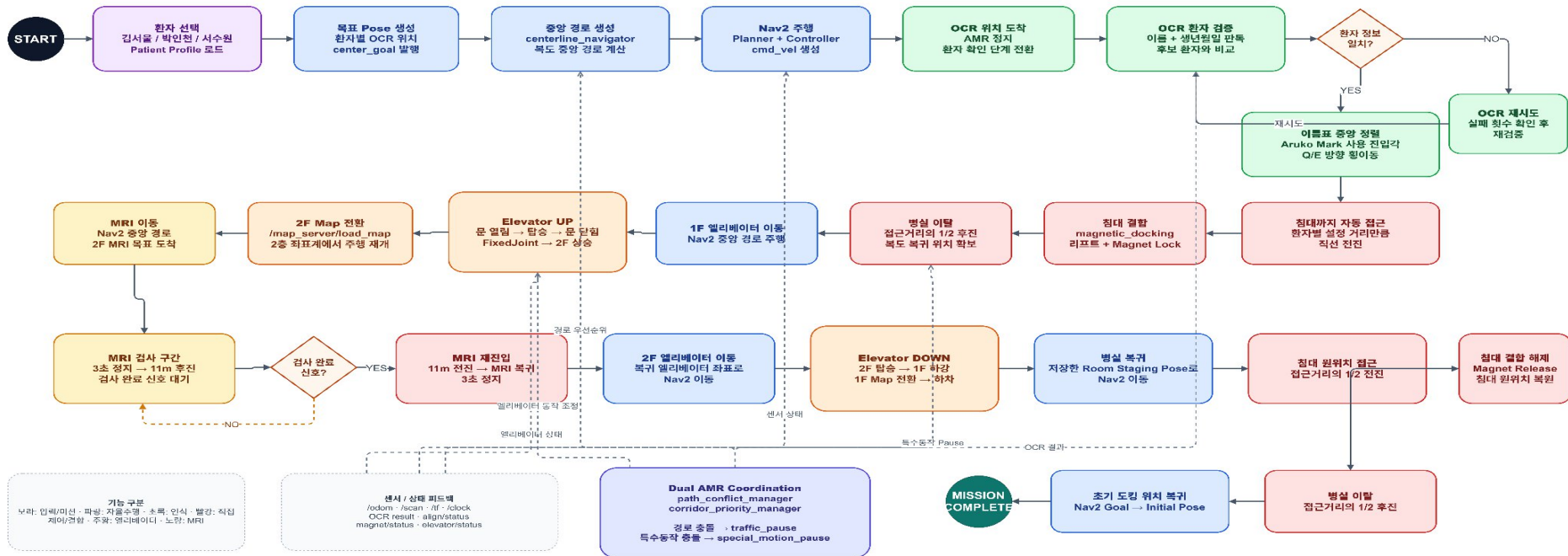


04 프로젝트 수행 경과

Main Functional Flow chart

병원 AMR 프로젝트 Functional Flow Chart

기능 중심 흐름 : Input → Recognition/Decision → Navigation/Control → Action → Feedback



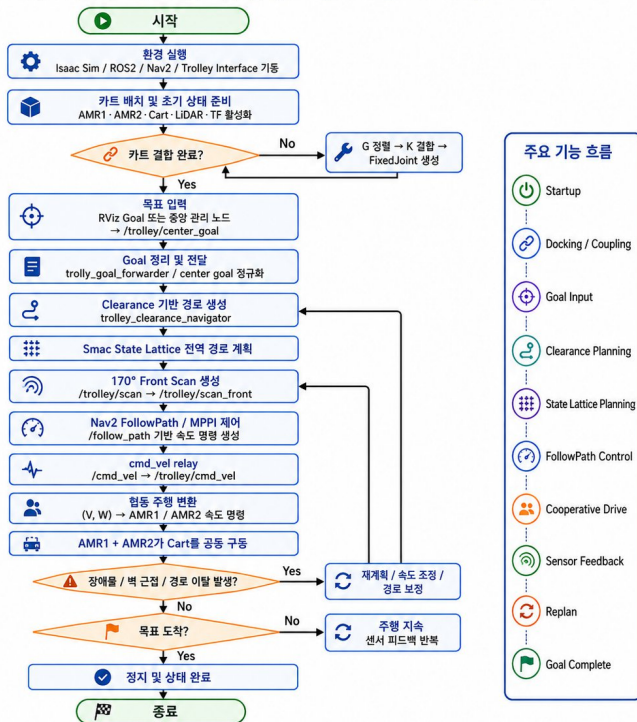
※ Functional Flow는 '어떤 기능이 어떤 입력을 받아 어떤 처리와 제어를 거쳐 다음 기능으로 전달되는가'를 중심으로 구성했습니다.

04 프로젝트 수행 경과

Sub Functional Flow chart

CART V5.17 Functional Flow Chart

hospital_bed_amr_FINAL_cart_v5_17_EDGE_170FOV_FEEDBACK_FLAT 기준



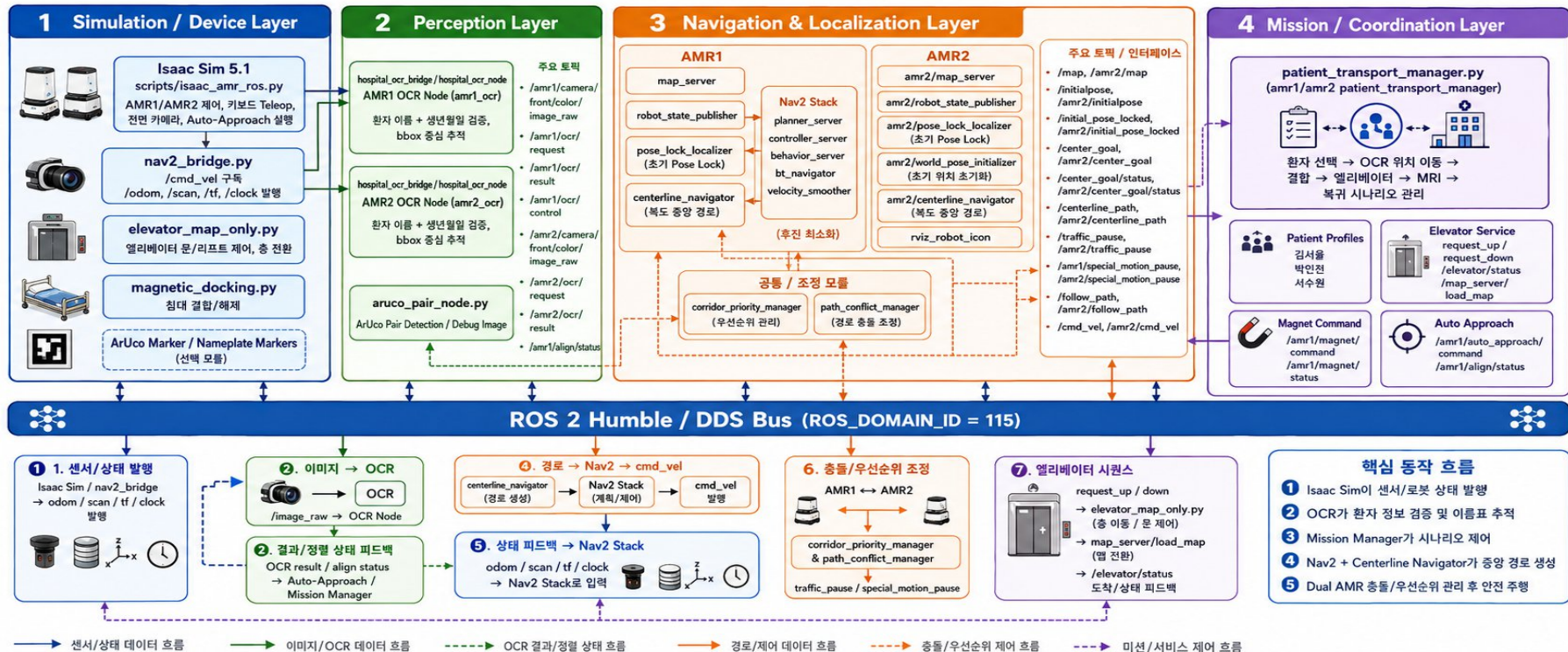
핵심 기능



04 MAIN Node Architecture

병원 AMR 프로젝트 Node Architecture

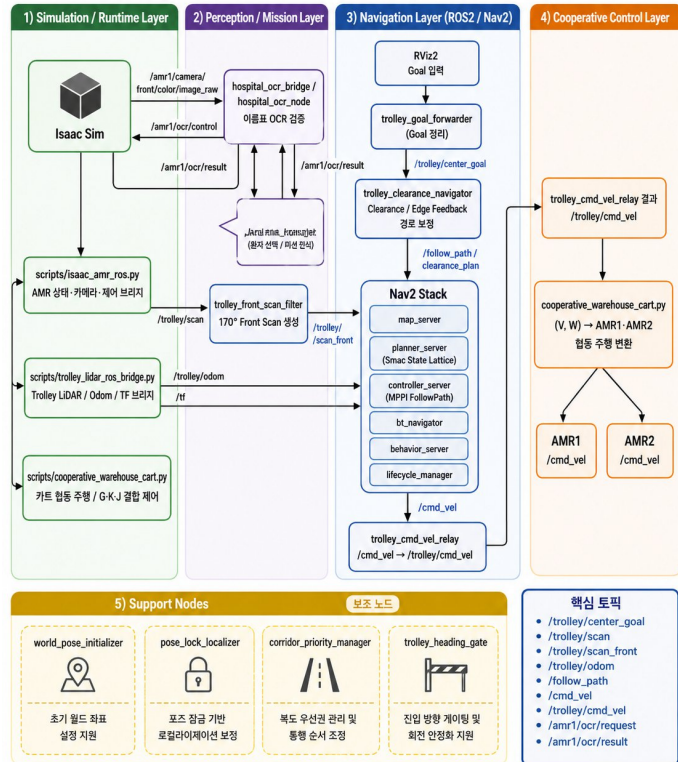
Isaac Sim + ROS 2 Humble + Nav2 + OCR + Elevator + Dual AMR Coordination



04 Sub Node Architecture

CART V5.17 Node Architecture

hospital_bed_amr_FINAL_cart_v5_17_EDGE_170FOV_FEEDBACK_FLAT 기준

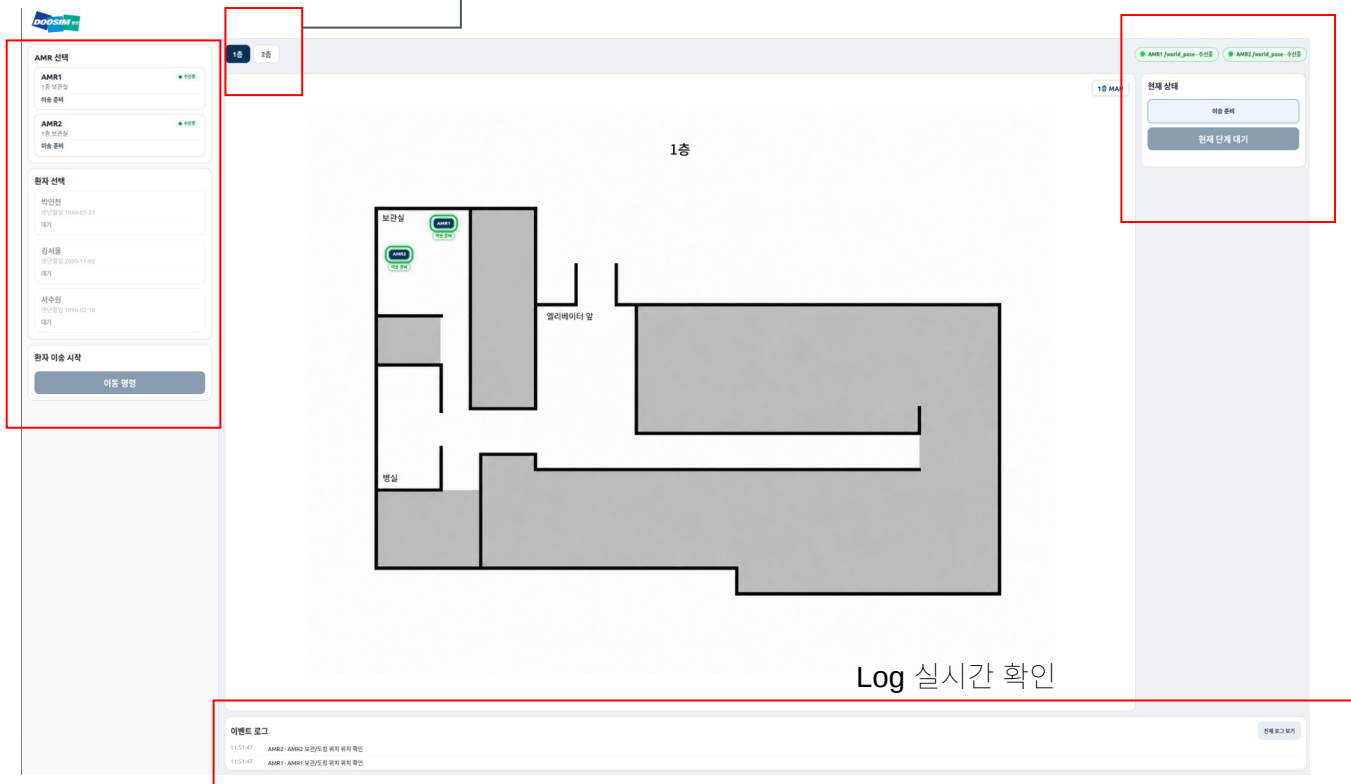


04. 프로젝트 수행 경과

간호사 인터페이스
환자 + AMR 선택창

층별 AMR 위치 실시간확인

MRI 실험 검사완료 버튼

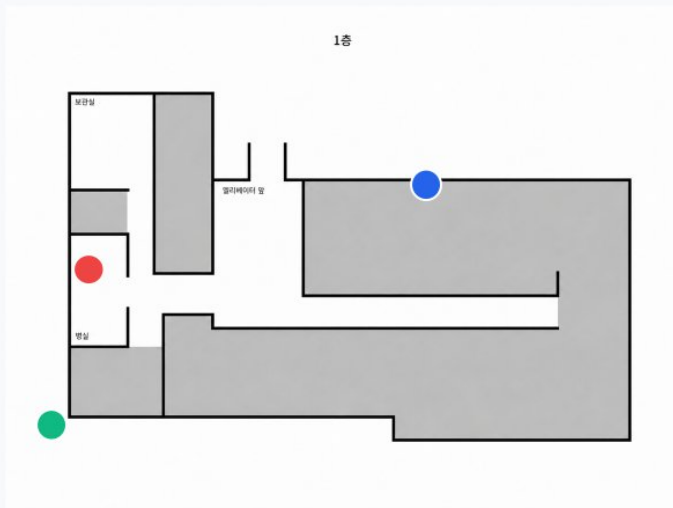


GUI 시연 영상

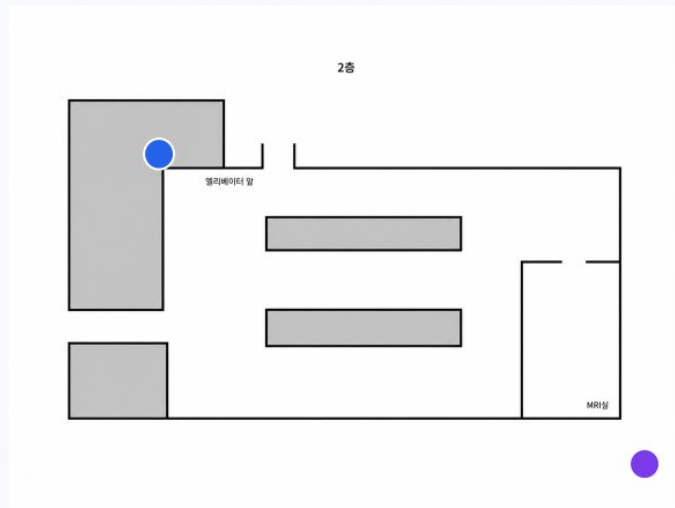
GUI DEMO VIDEO



GUI Map system



1층: 보관실 → 병실/OCR → 엘리베이터



2층: 엘리베이터 → MRI실 → 복귀

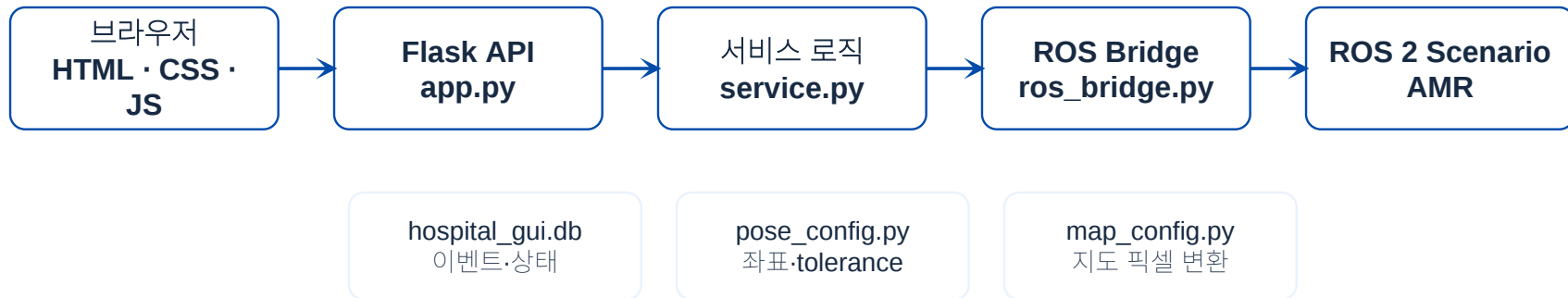
운영자는 AMR 선택 → 환자 선택 → 이동 명령 버튼으로 MRI 이송을 시작합니다.

화면의 AMR 위치는 임의의 애니메이션이 아니라 ``/amr1/world_pose`, `/amr2/world_pose`` 수신값을 기반으로 표시합니다.

이전/다음 좌표 버튼은 발표·디버깅용 화면 표시 기능이며 ROS 이동 명령을 보내지 않습니다.

02 GUI

전체 구조: 웹 **GUI**와 **ROS 2**를 분리



핵심 설계

웹 화면은 가볍게 유지하고, 실제 로봇/시나리오와의 연결은 **RosManager**가 담당합니다. 그래서 GUI는 발표용 시각화와 실제 **ROS** 연동을 동시에 처리할 수 있습니다.

브라우저 → **API** → 서비스 → **ROS Bridge** → **ROS 2 Scenario**

02 GUI

프론트엔드: 지도 위에 **AMR** 상태를 계속 렌더링



templates/index.html
화면 뼈대

static/app.css
대시보드 디자인

static/app.js
1초 상태 조회 · 마커 렌더

GET /api/state

→ 현재 AMR/환자/로그/지도 상태

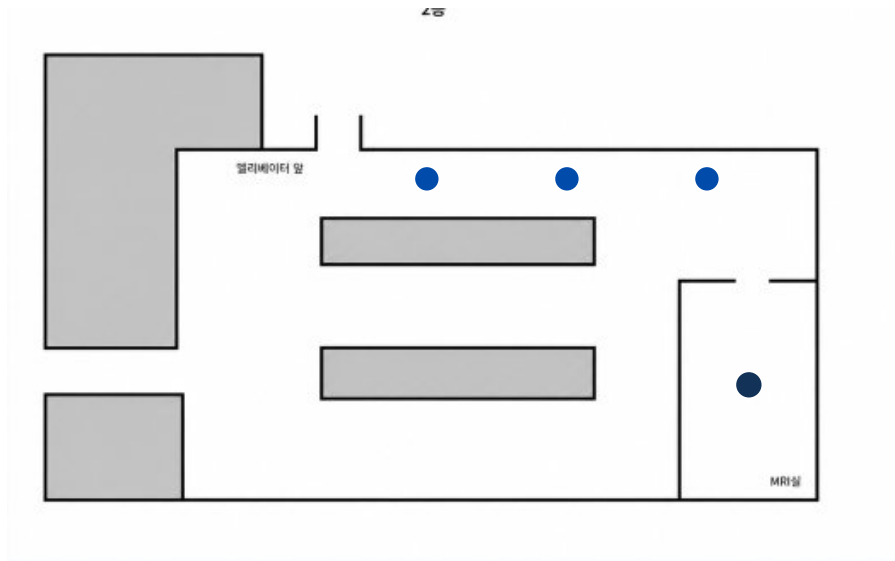
POST /api/command

→ START_MRI 또는 RETURN_TO_STORAGE

특징: 화면은 1초마다 서버 상태를 받아 갱신하고, 좌표 ON/OFF와 디버그 버튼은 발표 중 흐름 설명에 사용됩니다.

02 GUI

백엔드 핵심: 실제 좌표를 화면 좌표로 바꾸는 로직



1) pose_config.py
ROS 좌표와 현장 환경을 한 곳에서 관리

2) map_config.py
ROS x/y 를 지도 이미지의 u/v 위치로 변환

3) service.py
등록점 반경 안이면 스냅, 밖이면 경로 위 연속 이동으로 표시

registered points

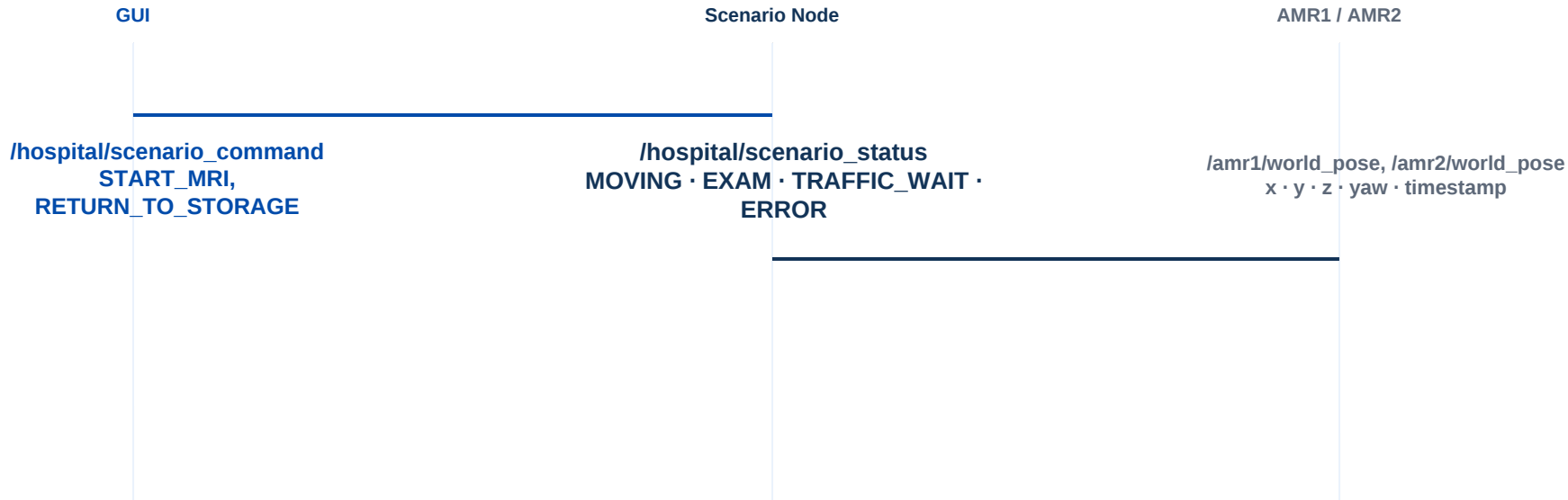
raw world_pose 투영

tolerance 내부 스냅

```
if distance <= tolerance:
    marker = nearest_registered_point
else:
    marker = projected_world_pose
```

02 GUI

ROS 2 연동: 토픽은 JSON 문자열로 단순화

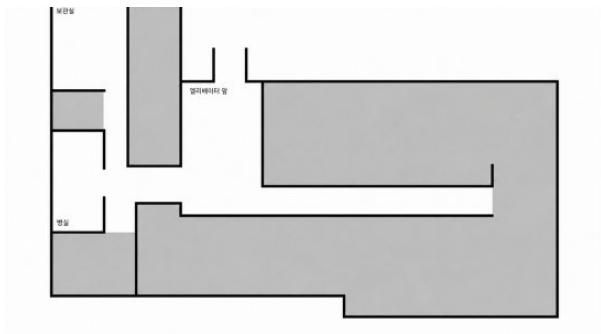


QoS = BEST_EFFORT / VOLATILE / KEEP_LAST(10)
GUI_POSE_SAMPLE_INTERVAL_SEC = 1.0

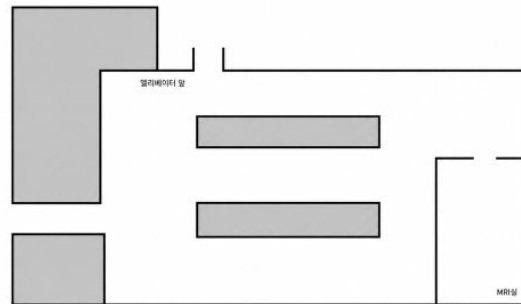
정리: 명령은 GUI → 시나리오로 보내고, 위치는 AMR → GUI로 받습니다. 그래서 화면 표시와 실제 주행 제어의 흐름을 분리합니다.

02 GUI

시나리오 흐름: 발표자가 설명하기 쉬운 **11**단계 표시



1층 경로



2층 MRI 경로

`service.py`는 현재 위치가 어느 구간에 있는지 판정해서 “현재 상태” 문구와 진행 단계를 만듭니다.

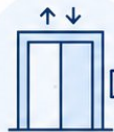
`static/app.js`는 이 상태를 받아 카드, 지도 마커, 이벤트 로그, 단계 배지를 다시 그립니다.

목표 과제 (Challenge)



다층 환경 자율주행 · 맵 전환

1F·2F 맵이 달라지는 환경에서 엘리베이터 전후 위치와 맵을 자연스럽게 전환



엘리베이터 기반 층간 무인 이동

AMR이 엘리베이터까지 이동한 뒤 탑승 · 층 이동 · 하차를 자동 수행



2대 AMR 충돌 회피 · 협업 주행

좁은 복도에서 경로 중첩 시 우선순위를 판단해 대기 또는 안전 회피



AI Vision 기반 환자 · 병상 식별

OCR로 이름·생년월일을 판독해 올바른 병상인지 검증하고 인식 오차를 보완



병상 결합 후 안정 주행

침대 결합 후 달라지는 크기와 회전 특성을 반영해 도킹 정렬 · 회전 안정성 · 복도 중앙 주행을 확보



핵심

우리의 목표 과제는 ‘인식-결합-주행-충돌 회피-층간 이동’이 하나의 시나리오로 안정적으로 연결되도록 만드는 것입니다.

04 프로젝트 수행 경과

ASSET (MAP) 병원 디지털 트윈 환경



1층



2층

1층 병실/보관실과 2층 MRI실을 구성하고, 엘리베이터를 통해 복층 이동 시나리오를 구현한다.

04 프로젝트 수행 경과

ASSET (MAP) 병원 디지털 트윈 환경



04 프로젝트 수행 경과

ASSET (AMR)



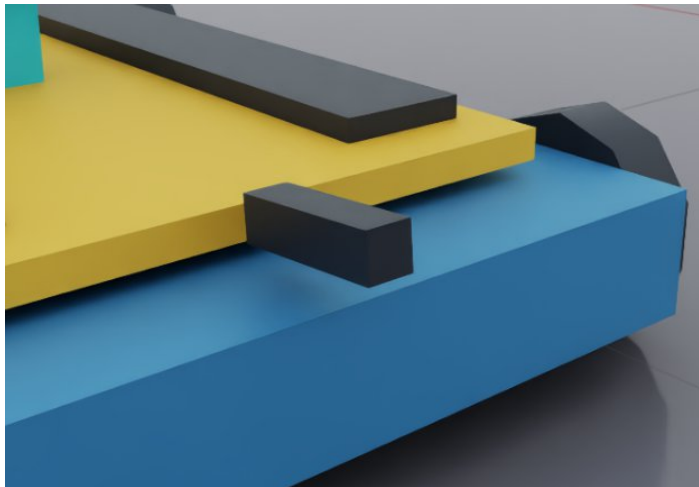
최종 적용 기준

- 자체 제작 옴니휠 AMR
- Nav2 연동을 위해 전진·후진·회전만 사용
- 360° LiDAR 1대/AMR
- RGB 카메라 적용

옴니휠이지만 자유 횡이동 자율주행은 사용하지 않으며, 침대 결합 안정성을 위해 제한된 주행 방식을 사용한다.

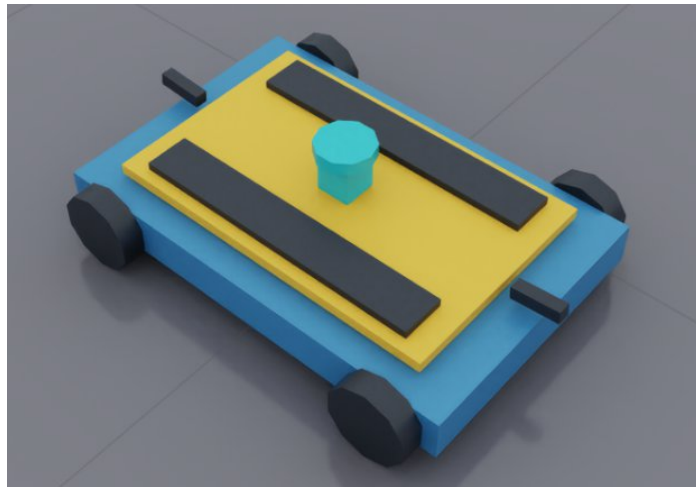
04 프로젝트 수행 경과

ASSET (AMR)



Isaac Sim Camera Sensor (RTX-rendered RGB Camera)

이름표 **OCR**과 환자 정보 확인
카메라 중심 기준 **X**축 정렬



**2D PhysX LiDAR
&
LIFT**

AMR 1대당 360도 LiDAR 1대
SLAM/Nav2와 장애물 감지에 사용

04 프로젝트 수행 경과

ASSET (AMR)



침대 결합

결합 순서

- AMR이 침대 하부로 진입
- 리프트 상승
- 자기식 **Fixed Joint** 생성
- 침대 결합 상태로 **Nav2** 이동
- MRI실에서 리프트 하강 후 **Joint** 해제

04 프로젝트 수행 경과

Nav2와 Pose Lock

Nav2 적용

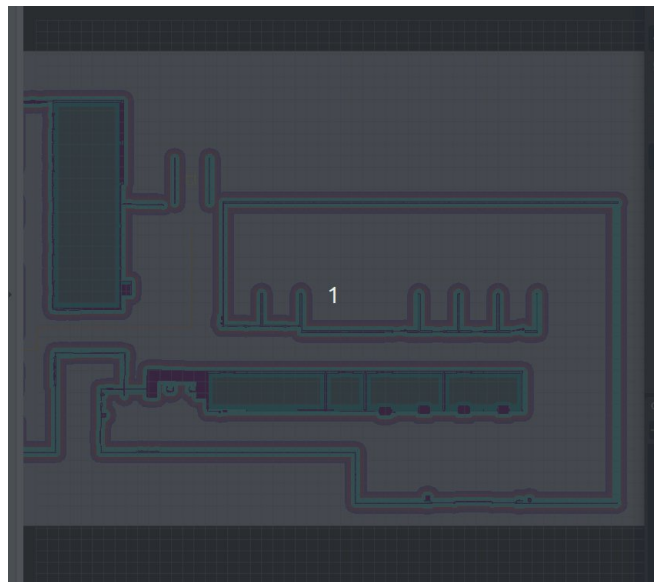
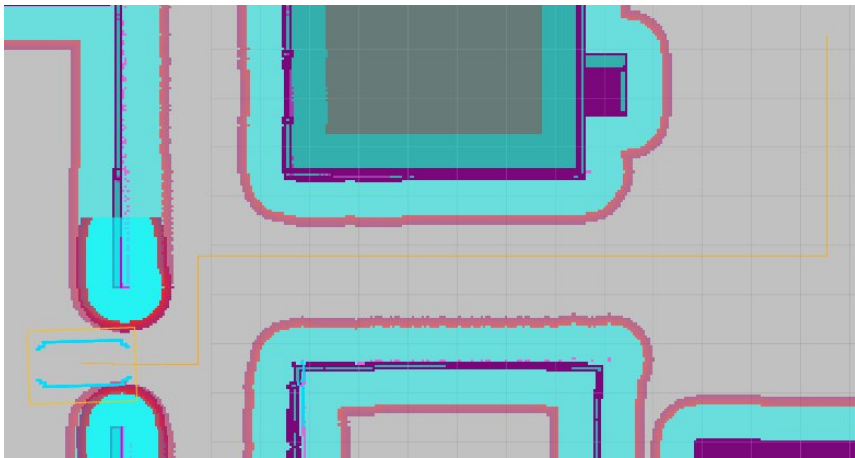
Occupancy Map 기반 목적지 이동과 FollowPath를 수행한다.

Pose Lock

초기 위치를 기준으로 map → odom 변환을 고정해 시뮬레이션 내 위치 기준을 맞춘다.

지도 전환

엘리베이터 이동 후 1층 지도에서 2층 지도로 자동 전환한다.



04 프로젝트 수행 경과

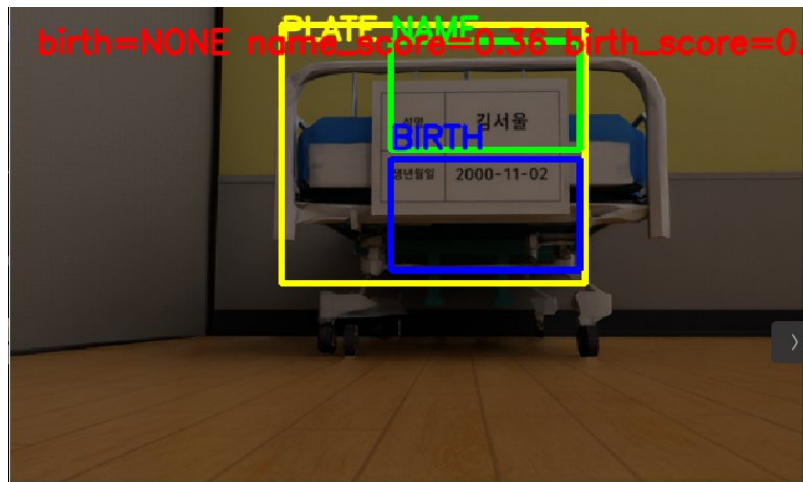
OCR 환자 검증

환자 데이터

- AMR1: 박인천
- AMR2: 김서울
- 추가 환자: 김인천
- 이름 + 생년월일 기반 비교

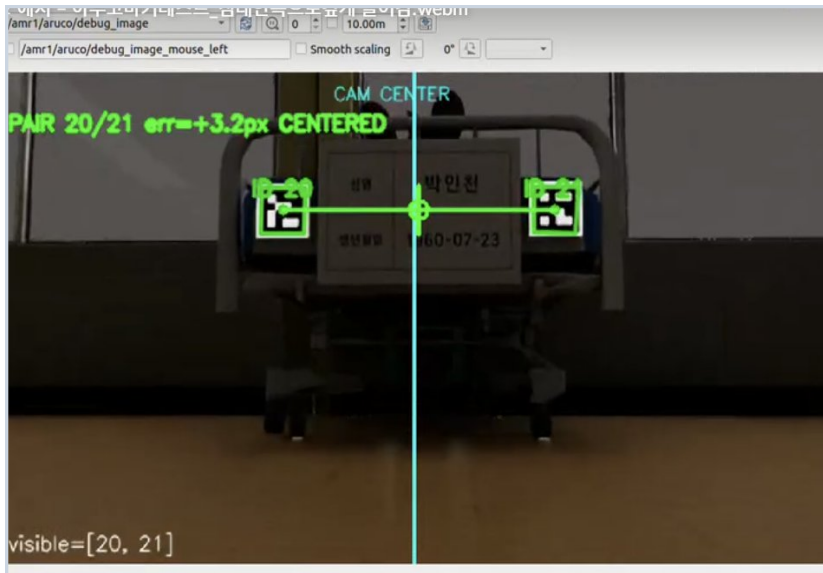
검증 흐름

- GUI 또는 미션 데이터에서 환자 정보 수신
- 침대 이름표 OCR 수행
- 이름·생년월일 비교
- 일치하면 침대 결합 단계 진행



04 프로젝트 수행 경과

Aruko mark



환자 침대의 이름표 아래에 부착하는 고유 식별 마커

AMR 전방 카메라가 마커를 인식하면 어느 환자 침대인지 빠르게 구분

마커마다 서로 다른 ID 번호를 가지고 있어 환자와 1:1로 매칭

OCR처럼 글자를 읽는 방식보다 조명·거리 변화에 비교적 강하고 인식 속도가 빠름

Key Issue

두개의 아루코 마크의 사이의 중심으로 진입각을 잡는다

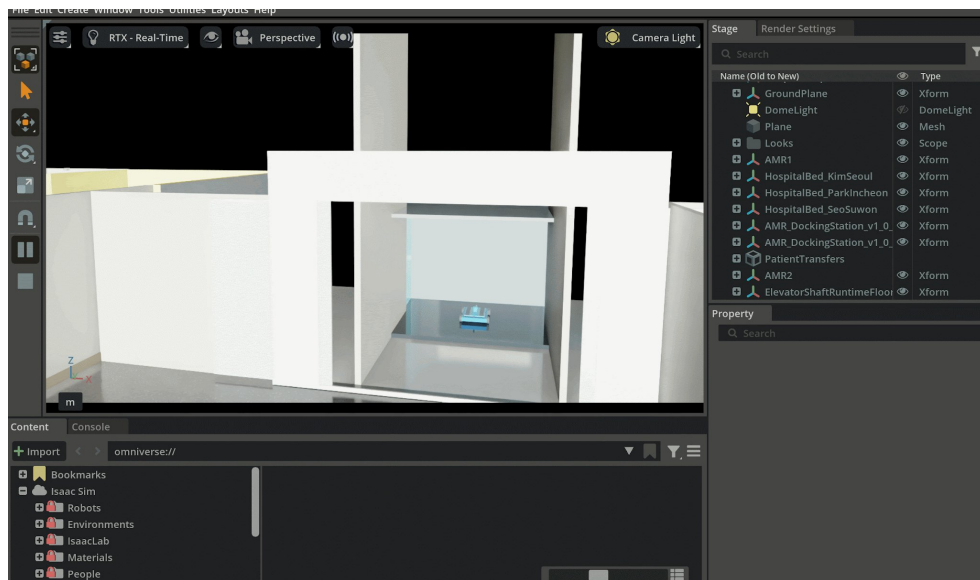
04 동적 FootPrint 프로젝트 수행 경과



침대 결합 후에는 **AMR** 크기만으로 경로를 계산하면 벽·코너와 충돌할 수 있으므로,
침대 크기를 반영한 풋프린트로 변경한다.

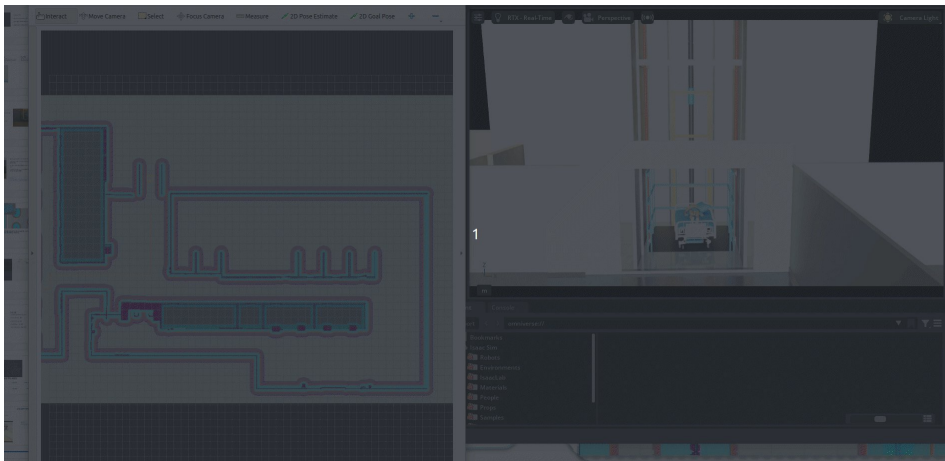
04 프로젝트 수행 경과

엘리베이터



04 프로젝트 수행 경과

엘리베이터 동작 과정



구현 기준

- AMR이 엘리베이터 앞 도착
- 도착 상태 통신 전송
- 엘리베이터 문 열림
- AMR 탑승 후 문 닫힘
- 2층 이동 및 지도 자동 전환

04 프로젝트 수행 경과

MRI실 검사과정



MRI실 침대 하차·재결합

1 MRI실 진입

2 침대 배치

3 환자 이동

4 AMR 후진 이탈

5 검사 완료 신호

6 AMR 재진입

7 환자 탑승

8 복귀

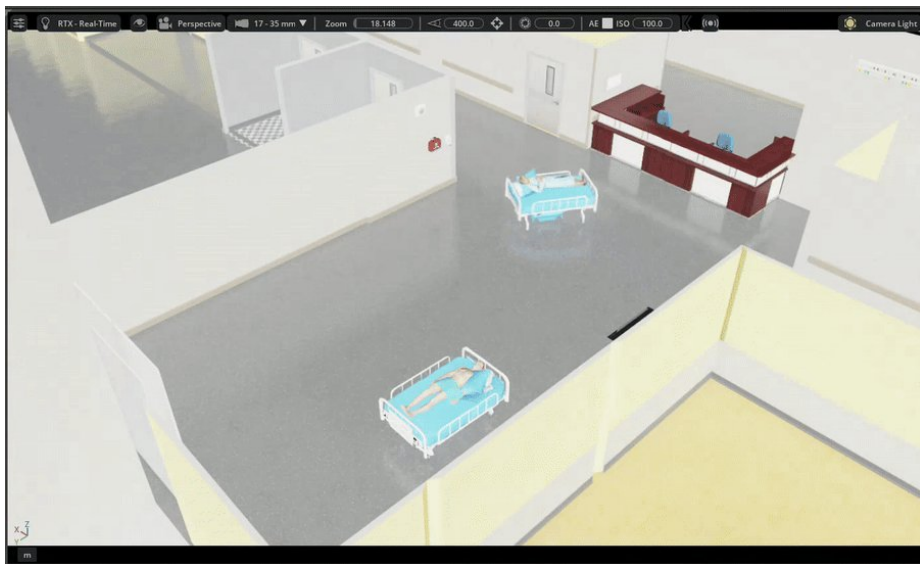
04 프로젝트 수행 경과 충돌회피 모션 & 설계

AMR1

김서울 환자 검사완료 후 병실 이동

AMR2

박인천 환자 MRI이송



2대 AMR 운영 계획 충돌회피

충돌 방지

MRI 검사 우선권 가진 AMR2를 위해 AMR1은
안전 구역에서 대기 후 출발

04 프로젝트 수행 경과

예외처리

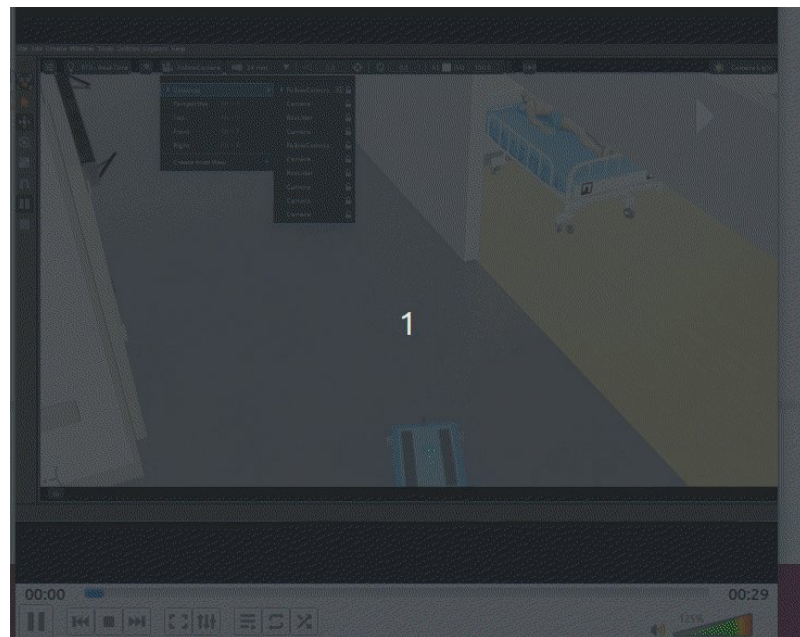
환자가 다르거나 침대가 없는 경우

AMR1

박인천 환자 MRI이송

도킹 스테이션 복귀

신원 불일치 확인 후 도킹 스테이션 복귀



04 프로젝트 수행 경과

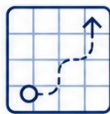
알고리즘 요약

Main Mission



Pose Lock

초기 위치를 자동 고정해
주행 기준 좌표를 안정화



중앙 경로 자율주행

이동경로의 가까운 벽 A*와 벽 이격
비용으로복도 중앙 경로 생성



OCR + ArUco 인식

이름표·생년월일·마커 ID를
이용해 환자를 확인



X축 정렬 · 침대 결합

시각 오차를 줄인 뒤
Lift + Magnetic Fixed Joint로 결합



듀얼 AMR 충돌 조정

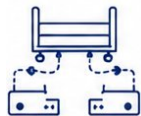
경로 중첩 구간을 판단해
우선권과 일시정지를 제어



엘리베이터 · 맵 전환

층 이동 후 상태머신 기반으로
1F/2F 맵을 전환

Sub Mission



트롤리 협동 이송

두 AMR이 트롤리를 하나의
이동체로 결합해 협동 이송



MPPI 경로 최적화

State Lattice 경로를
여유폭기반으로 보정해 안전 주행

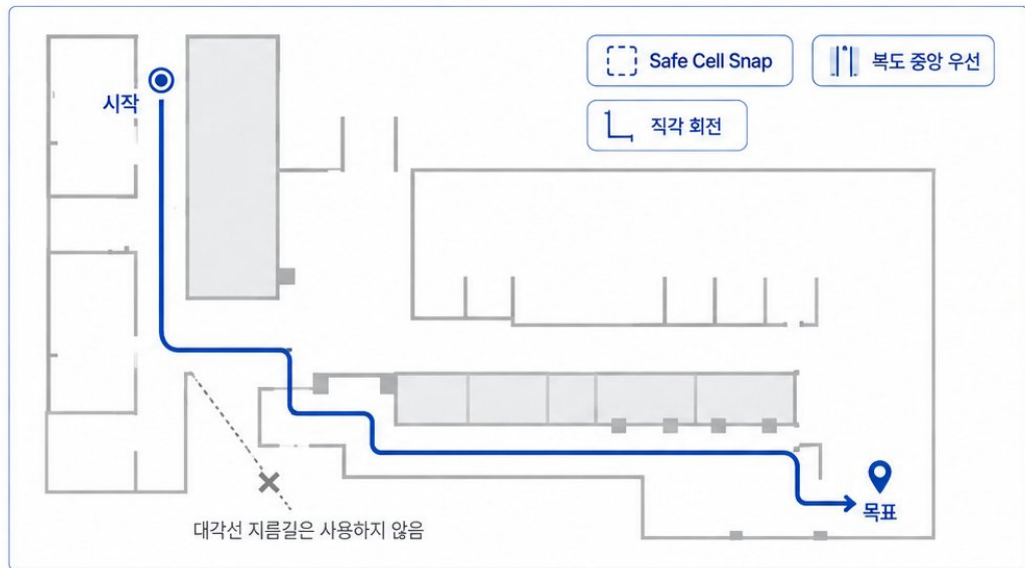
핵심

인식 → 정렬 → 결합 → 중앙 경로 주행 → 교통 제어 → 층간 이동이 하나의 시나리오로 연결됩니다.

04-1 핵심 알고리즘

중앙 경로 자율주행 알고리즘

벽에 붙는 최단경로 대신 복도 중앙과 직각 회전을 우선



1. 시작·목표 보정

현재 위치와 목표 좌표를 안전한 셀로 보정해 출발점과 도착점을 안정화



2. 4방향 A* 경로 생성

대각선 대신 4-connected A*를 사용하고, 벽 이격 비용과 회전 패널티를 함께 반영



3. Segment별 주행

경로를 직선 구간으로 분할한 뒤
코너에서 정지·회전하고
Nav2 FollowPath로 각 구간을 수행

$$\text{Cost} = 1 + \text{Center Cost} + \text{Turn Cost}$$

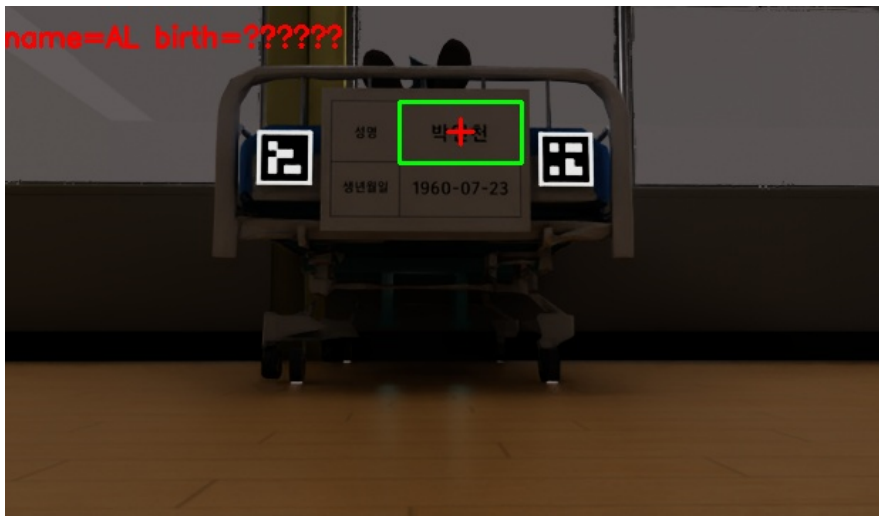
$$\text{Center Cost} = w / \text{clearance}^2$$

효과: 벽에서 멀수록 유리, 불필요한 회전은 억제

04-2 핵심 알고리즘

환자 인식 · 정렬 · 침대 결합 알고리즘

OCR + ArUco + Lift + Magnetic Fixed Joint



실제 OCR/ArUco 디버그 화면 예시



1. OCR 요청

RGB 카메라 영상에서 이름표 영역을 추출하고 이름·생년월일을 판독



2. ArUco 쌍 인식

좌·우 마커 ID 조합으로 환자를 식별하고, 중심 오차와 yaw 오차를 계산



3. X축 중심 정렬

pair center와 camera center 차이를 이용해 X축 중심을 맞춘 뒤 접근



4. 침대 결합

Lift로 높이를 맞춘 후 Magnetic Fixed Joint를 생성해 침대를 안정적으로 연결

환자 ID 예시

김서울: 10 / 11
박인천: 20 / 21
서수원: 30 / 31

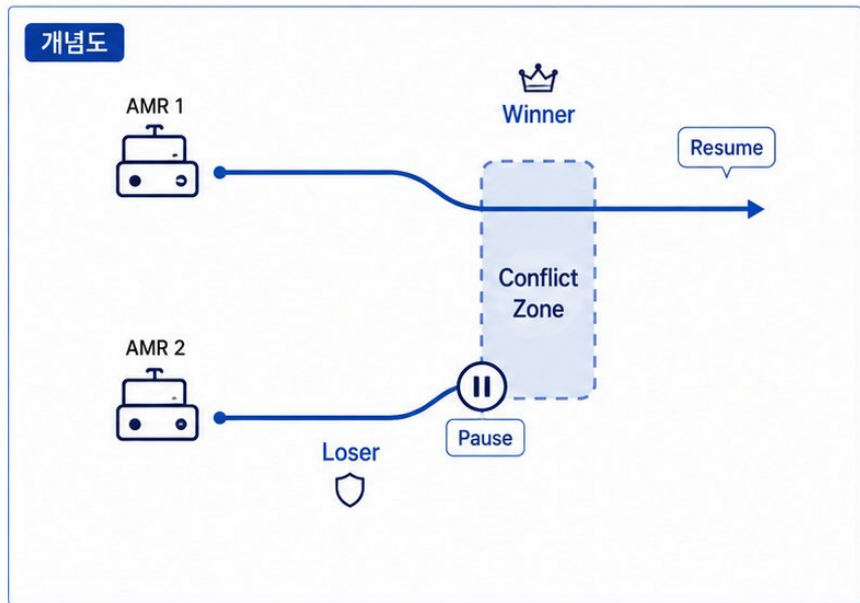
효과

- 오인식 감소
- 접근 정밀도 향상
- 결합 안정성 확보

04-3 핵심 알고리즘

듀얼 **AMR** 경로 충돌 조정 알고리즘

두 AMR의 path overlap을 분석해 우선권과 일시정지를 제어



규칙 기반 로직



1. 경로 수집

각 AMR의 centerline path와 현재 위치를 지속적으로 수집



2. 충돌 구간 탐지

미래 경로 점들이 일정 거리 이내로 가까워지면 conflict zone을 생성



3. 우선순위 판단

특수 동작(OCR-결합-엘리베이터) > 이미 구간에 진입한 AMR > 진입까지 거리가 더 짧은 AMR > 충돌 시 AMR1



4. 양보 및 재개

양보 AMR만 일시정지시키고, 우선 AMR 통과 후 자동으로 resume



보조 안전 로직

실제 world pose 기반 거리도 함께 확인해 근접 상황을 한 번 더 감시

04-4 핵심 알고리즘

엘리베이터 · 맵 전환 알고리즘

도착 통신과 상태머신 기반의 층간 이동 제어



04-5 핵심 알고리즘

MPPI 연동 경로 최적화 알고리즘

State Lattice raw path를 clearance-balanced path로 보정한 뒤 FollowPath로 수행



입력

Static Costmap + Full 360° LiDAR Scan

전역 지도는 벽/고정 장애물을 제공하고, 360° 스캔은 동적 장애물과 현재 clearance를 보조합니다.

/global_costmap/costmap

/trolley/scan

/trolley/raw_plan

/trolley/clearance_plan

목표

큰 트롤리 footprint가 복도 벽에 붙지 않도록 경로 자체를 보정

최단거리보다 안전 여유, 좌우 균형, 회전 가능 공간을 우선해 실제 주행 중 벽 간섭과 코너 걸림을 줄입니다.

좌·우 clearance 균형

yaw 보존

swept footprint 검증

조건부 재계획

핵심

State Lattice가 만든 주행 가능 경로를 DP clearance optimizer가 한 번 더 보정해 MPPI 추종 안정성을 높입니다.

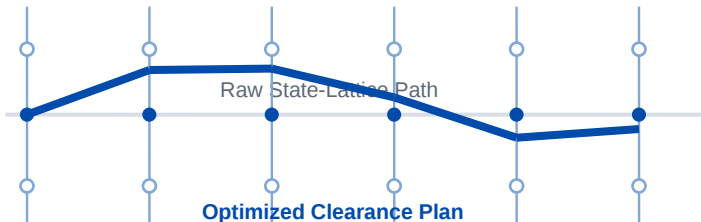
04-5-1 핵심 알고리즘

DP Clearance Optimizer

각 경로 샘플마다 **lateral candidate**를 만들고 전체 시퀀스를 동적 계획법으로 선택

개념도

Left wall



Right wall

candidate_step=0.10m

max_shift=1.20m

max_shift_step=0.15m



1. 후보 생성

raw sample의 normal 방향으로 lateral shift 후보를 생성



2. 단일 후보 비용 계산

좌/우 clearance, inflation, deviation, corridor 중심성을 점수화



3. DP 전이 비용 적용

shift 변화량·shift 가속도·추가 경로 길이를 패널티로 반영



4. Swept Footprint 검증

선형 이동과 yaw 변화 사이의 전체 물체 충돌을 확인

Score = balance + min_clearance + inflation + deviation + smoothness + extra_length

핵심

한 점씩 욕심내서 고르는 방식이 아니라, 전체 경로의 **lateral shift** 흐름을 **DP**로 최적화합니다.

04-5-2 핵심 알고리즘

조건부 재계획 · 최종 회전 안전 로직

실제 주행 중 scan/clearance feedback을 확인해 필요한 순간에만 경로를 다시 계산

재계획 Trigger



A. 새 장애물 감지

full-scan obstacle이 남은 swept route와 겹치면 카운트



B. Path 이탈 감지

현재 trolley center와 optimized path의 cross-track 오차 확인

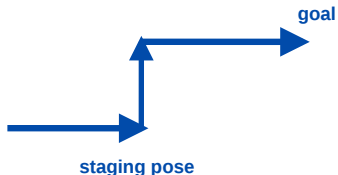


C. 벽 여유 부족

front/left/right/rear edge clearance가 기준 이하인지 확인

※ 일회성 오검출 방지: 여러 번 연속 확인 후 FollowPath cancel → 재계획

Final Yaw Staging



목표 지점에서 바로 회전하면 벽과 가까운 경우, 목표 뒤 staging pose에서 먼저 회전 후 직선 진입합니다.

FollowPath 수행



Rotation Shim

초기 진행 방향과 yaw 차이가 크면 먼저 회전 정렬



MPPI Controller

최적화된 path를 따라 local trajectory를 샘플링·평가하며 추종



Fail-safe Fallback

DP 후보가 과제역되면 raw lattice path를 표시하고 안전성 재검증

/trolley/clearance_plan → /follow_path → controller_server

04-6-1 핵심 알고리즘

트롤리 협동 이송 알고리즘 요약

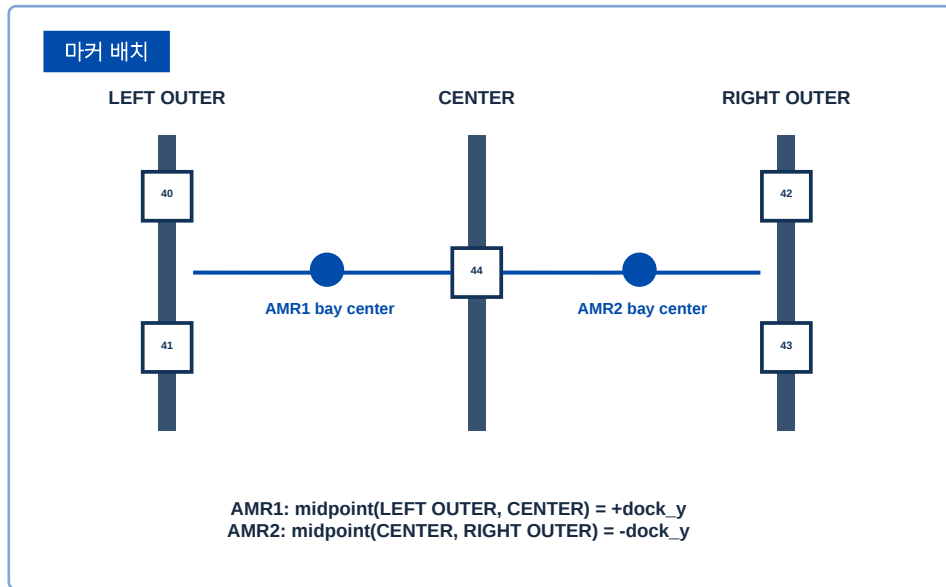
두 AMR이 트레이를 하나의 협동 이동체로 인식·도킹·운반



04-6-2 핵심 알고리즘

3-Post ArUco Gate 도킹 알고리즘

outer marker와 center marker의 중점을 각 AMR의 실제 dock bay 중심으로 사용



1. Virtual Outer Marker



상·하 outer marker가 모두 보이면 중점을 평균하고, 하나만 보여도 fallback tracking

2. Pair Midpoint 계산



outer와 center marker의 pair_center_x를 계산해 camera center와 비교

3. Final Ingress Steering



center_error_px를 tray_cmd_vel의 미세 조향 입력으로 사용

4. SINGLE ID Fallback



PAIR가 완성되지 않아도 기대 marker 하나가 보이면 fixed insert로 데모 진행

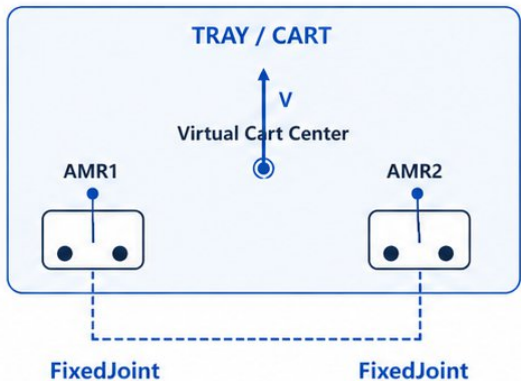
ID 기준: AMR1 = 40/41 + 44, AMR2 = 44 + 42/43, Dictionary = DICT_4X4_50

04-6-3 핵심 알고리즘

Dual FixedJoint · 협동 속도 분배 알고리즘

트레이 중심 $\text{twist}(V, \omega)$ 를 두 AMR의 위치에 맞는 개별 속도로 변환

개념도



발표용 핵심 문장

두 AMR을 따로 움직이지 않고, 트레이 중심에 V 와 ω 를 주면 각 AMR의 위치에 맞게 속도를 나눕니다.

1. 실제 도킹 오프셋 측정

FixedJoint 체결 후 cart frame 기준 AMR1/AMR2의 실제 local offset을 측정

Σ

2. 중심 twist → 개별 AMR 명령

직진은 동일 선속도, 회전은 반경 차이에 따라 좌·우 속도 차등 적용

↻

3. Sync Assist

수동 caster drag와 joint catch-up을 줄이기 위해 cart rigidbody에도 중심 twist 를 보조 적용

직진: $\omega=0 \rightarrow$ 두 AMR 동일 V | 회전: $\omega \neq 0 \rightarrow$ 안쪽 느리게, 바깥쪽 빠르게

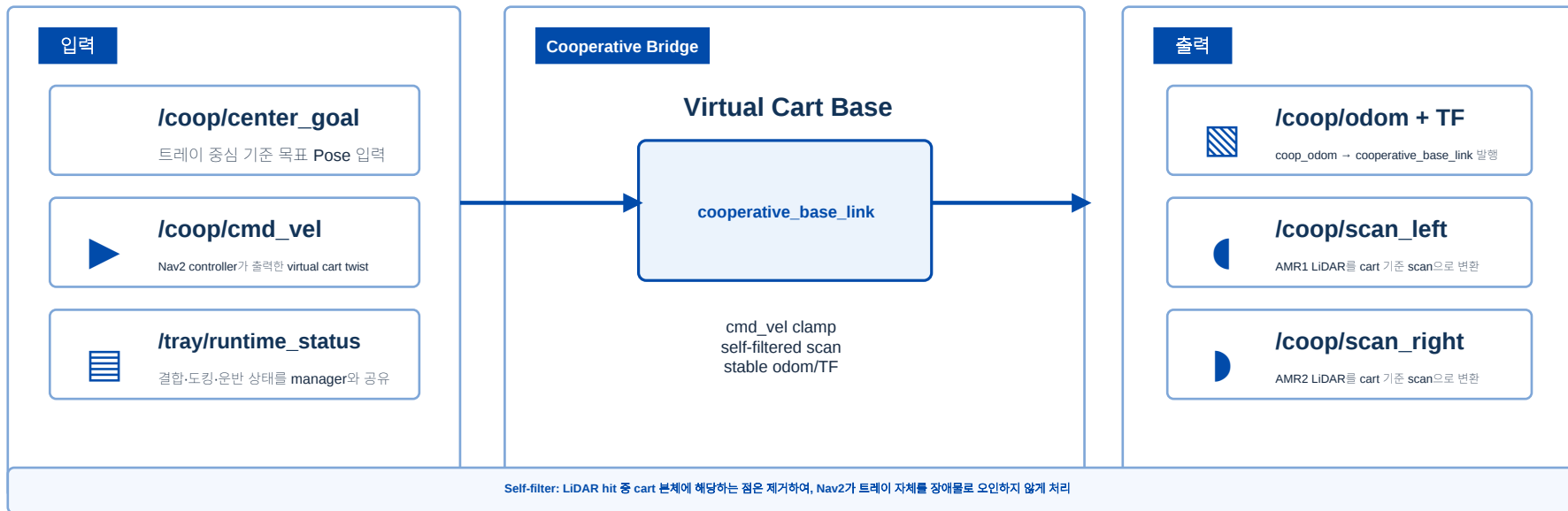
핵심

협동 제어의 목표는 “두 대 같은 속도”가 아니라, 같은 각속도로 하나의 강체처럼 회전하도록 속도를 분배하는 것입니다.

04-6-4 핵심 알고리즘

Cooperative Nav2 Bridge · 360° LiDAR 운용

결합 이후 트레이 전체를 하나의 **virtual robot**으로 Nav2에 연결



대형 트롤리 협동 운송 구조 설계

2대 AMR을 각각 제어하는 방식에서 트롤리 중심의 하나의 이동체로 제어 기준을 전환

기존 방식

AMR1·AMR2를 각각 따로 제어

- 속도 차이·방향 오차 발생
- 회전 시 트롤리 비틀림



제안 구조

트롤리를 기준 객체로 설정

- 중심 선속도 V , 각속도 ω 만 생성
- 두 AMR이 속도를 분담



핵심 | 개별 AMR이 아니라 트롤리 중심을 움직이면 두 AMR이 하나의 강체처럼 함께 움직인다

AMR 협동 주행 시연 영상

AUTONOMOUS MEDICAL SUPPLY TRANSPORT

DUAL AMR COOPERATIVE TRAY SYSTEM



REAL WORLD LOCK & BRIDGING



DUAL AMR COOPERATION



MPPI + CLEARANCE NAVIGATION



SAFE & RELIABLE HOSPITAL DELIVERY

최고품질 제품지원을 | 동업 AMR협력 시스템 | MPPI 기반 자율주행

REALISTIC SIMULATION
REAL-WORLD TECHNOLOGY

V2.17.13 + MPPI
AUTONOMOUS SOLUTION



04 프로젝트 수행 경과

트롤리 장애물 주행

대형 트롤리 주행 경로 개선

기본 경로의 벽 여유 부족 문제를 확인하고,
Clearance 기반 보정 경로를 생성

빨간선

State Lattice 기본 경로

주행 가능하지만 한쪽 벽에 가까워질 수 있음

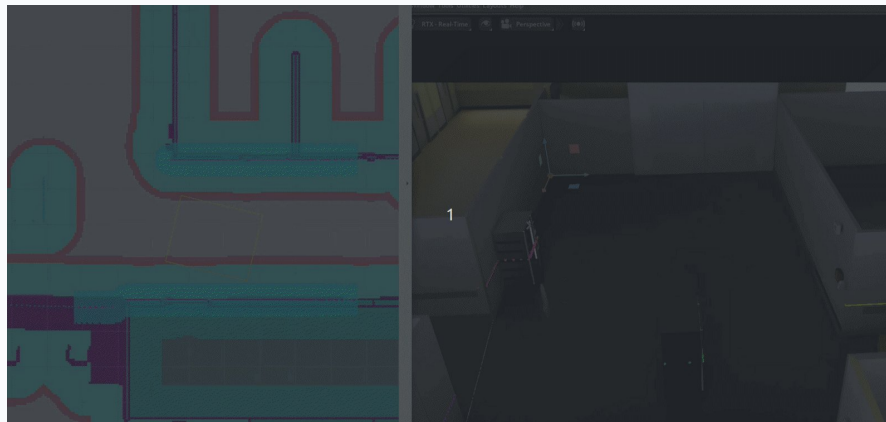
초록선

Clearance Optimized Path

좌·우 벽 여유를 비교해 안정적인 경로로 보정

빨간선 = 기본 경로

초록선 = 보정 경로



기본 경로의 좌·우 **Clearance**를 비교하여, 대형 트롤리가 벽과 충분한 여유를 확보하며 주행하도록 경로 보정 알고리즘을 구현하였다.

각속도 기반 속도 분배 원리

같은 트롤리를 회전시키려면 두 AMR은 같은 회전 속도를 유지하되, 이동 거리에 맞춰 선속도를 다르게 가져야 한다.

1 각속도 ω

방향이 얼마나 빠르게 바뀌는지 나타내는 값이다.
예: 1초 동안 몇 도 회전하는가

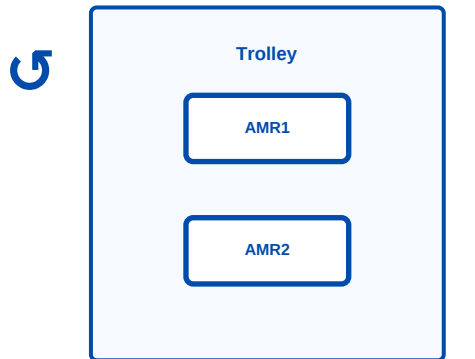
2 같은 ω 가 필요

트롤리가 비틀리지 않으려면 AMR1과 AMR2가 같은 각도로 동시에 돌아야 한다.

3 속도는 다르게

바깥쪽 AMR은 더 먼 원호를 지나므로 더 빠르게, 안쪽 AMR은 더 느리게 움직여야 한다.

우리 AMR에 적용한 회전 개념



바깥쪽 AMR = 더 빠르게

안쪽 AMR = 느리게

직진: 둘 다 같은 속도
회전: 반경에 따라 속도 차등

공통 각속도 ω

트롤리 중심 명령

`/coop/cmd_vel`

V = 앞으로 가는 속도
 ω = 회전하는 속도

$$AMR1 = V - \omega \cdot y$$

$$AMR2 = V + \omega \cdot y$$

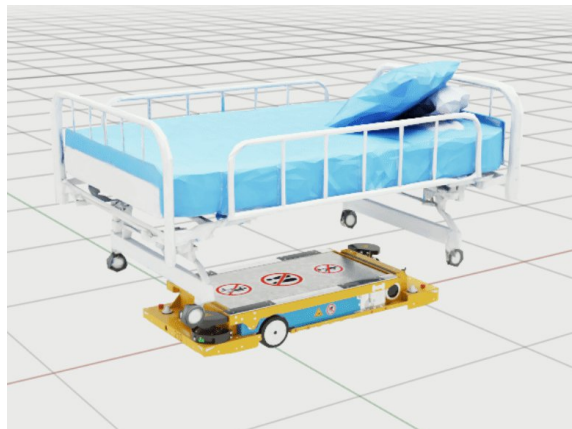
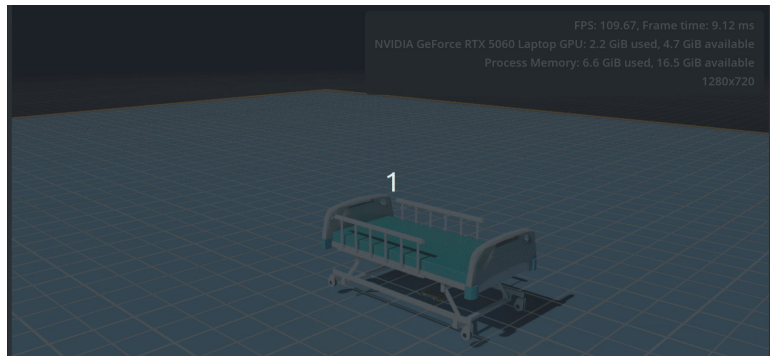
목표는 “같은 속도”가 아니라
“같은 각속도 + 다른 선속도”

04 프로젝트 수행 경과

Key Issue 1 - Collider와 침대 하부 공간

Issue

침대 GLB가 하나의 Mesh라 Convex Hull Collider 적용 시 침대 하부 빈 공간이 막힘



04 프로젝트 수행 경과

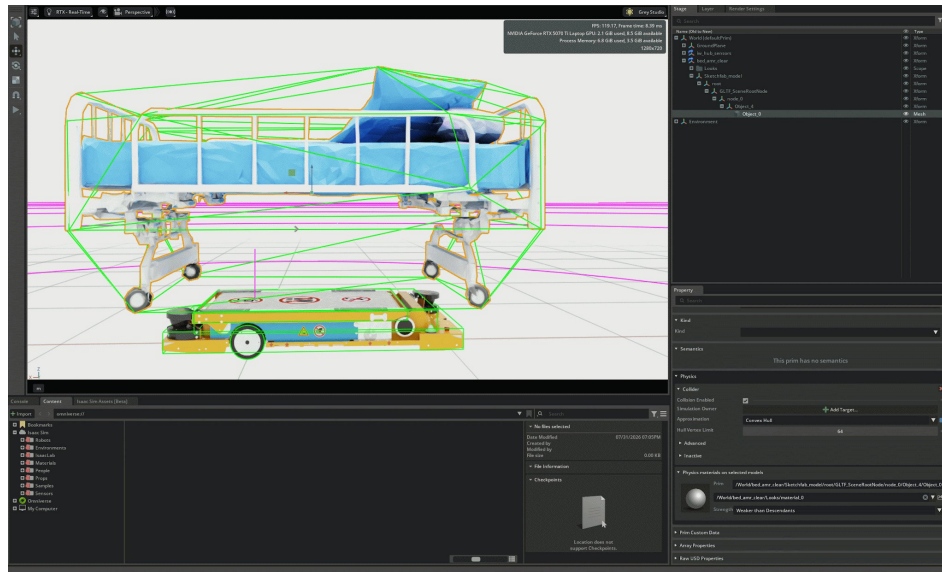
Key Issue 1 - Collider와 침대 하부 공간

Solution

Convex Decomposition으로 변경하여 침대 형상을
여러 Collider로 분리

Result

AMR이 침대 하부로 진입할 수 있고, 침대 튀어오름
문제를 완화



04 프로젝트 수행 경과

Key Issue 2 - AMR 선정 과정

Issue

- Isaac Sim 자체 제작 Asset 사용시 각종 제약 발생 (리프트 제한, 크기)

Solution

- AMR을 자체 제작함

Result

- Asset 을 사용했을 때의 제약이 사라짐



기본 에셋



자체 제작

04 프로젝트 수행 경과

Key Issue 3 - 맵 통합 (collision 이슈)

Issue

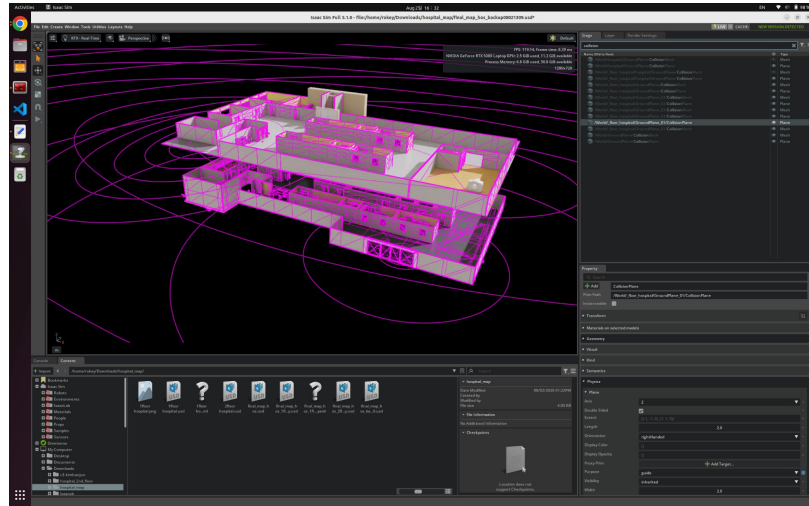
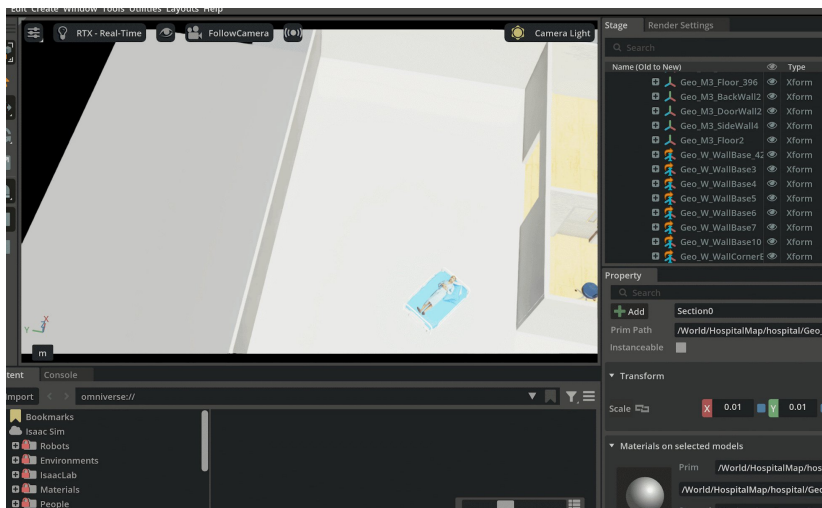
1층, 2층 map 구현시 2층 바닥제를 두장 설치 후 collision 적용 바퀴의 PhysX가 매 프레임 위치를 보정해서 물리 진동으로 화면이 지진이남

Solution

바닥 하나 제거 후 바퀴의 PhysX를 빼고 안전성을 위해 2F Map + Pose Reset로 2층 map 전환시 바닥위치로 기준 고정

Result

물리진동 삭제



04 프로젝트 수행 경과

Key Issue 4 - Nav2 시간, TF, LiDAR 안정화

Issue

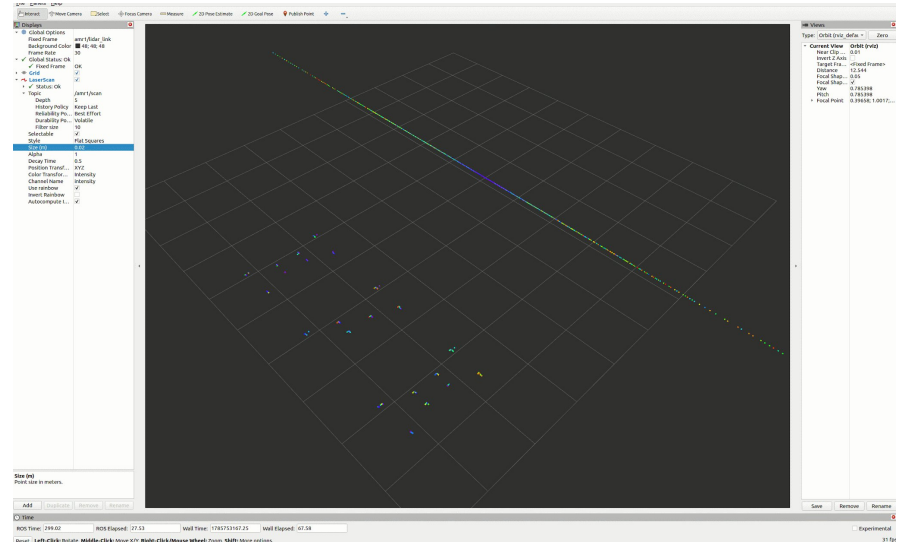
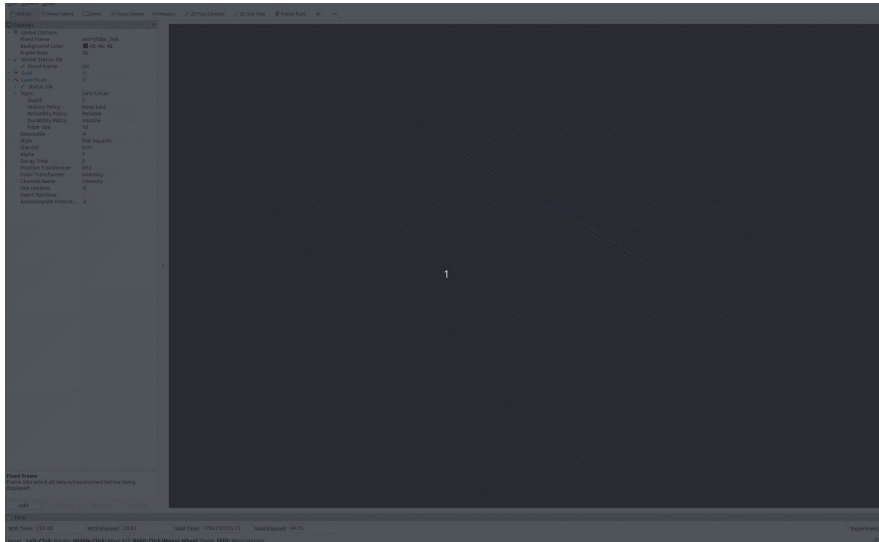
- RViz LiDAR 포인트 깜빡임
- Costmap 갱신 불안정
- 시뮬레이션 시간과 ROS 시간 불일치

Solution

- /clock 발행
- use_sim_time=true
- Timeline Loop 비활성화
- LaserScan Decay Time 조정

Result

- LiDAR 표시 안정화
- Nav2 센서 데이터 동기화
- AMR 경로 추종 가능



04 프로젝트 수행 경과

Key Issue 5 - NAV2

Issue

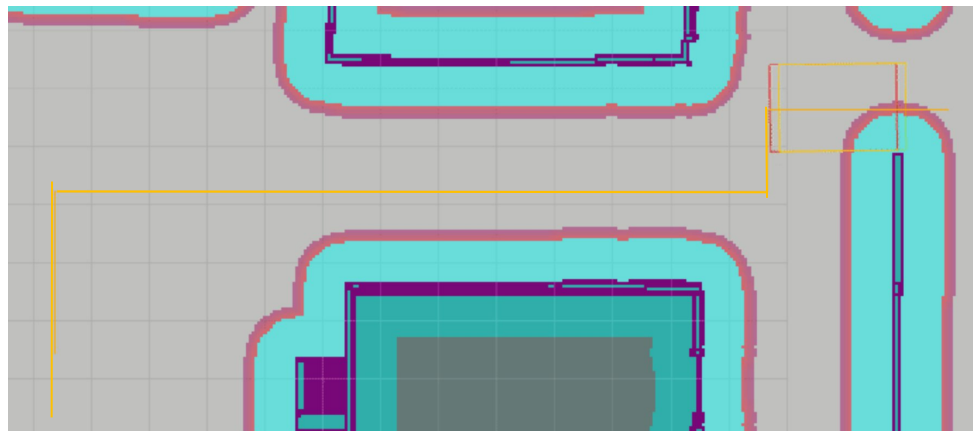
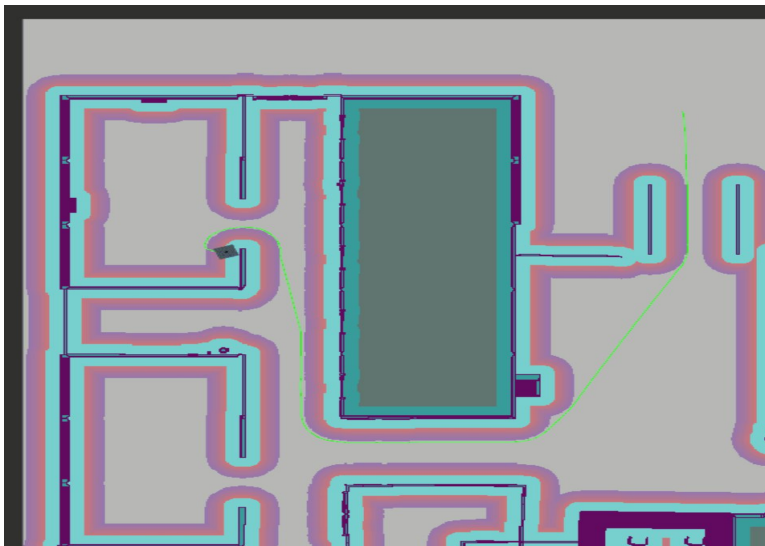
최단거리로 이동하는 자율주행 특성상 벽을 굽어서 가는 현상이 발생

Solution

비용함수를 사용하여 가까운벽에서의 비용을 증가시켜 경로상 최대한 중앙을 이용하게 설계

Result

경로상에서 벽에서 어느정도 여유가생겨 안정적인 운반가능



04 프로젝트 수행 경과

Key Issue 6 - 침대 바퀴 물리적 특성으로 인한 회전저항 증가

Issue

침대 바퀴 4부분의 물리적 적용으로 인해 회전 저항 증가하여 기존 회전속도와 각속도로는 회전불가능

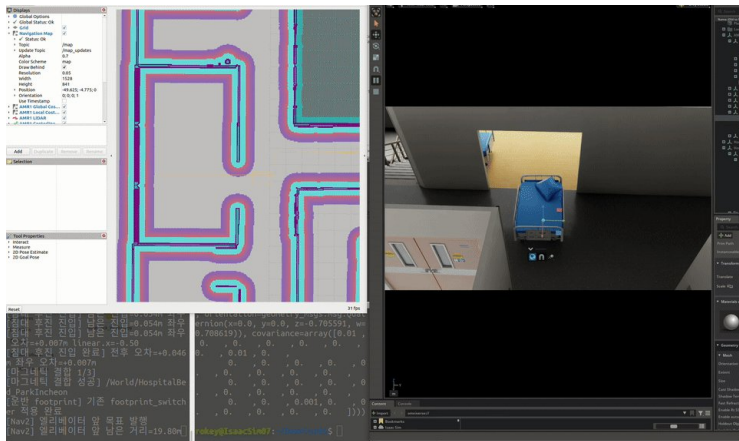
Solution

물리적 계산이 어렵고 구현하기에는 테스트 시간을 많이 소요할거같아

회전 구간에서는 Nav2에서 계산된 각속도 명령을 Isaac Sim의 AMR RigidBody에 직접 적용하여, 적재 상태와 관계없이 목표 회전 속도를 안정적으로 유지하도록 보완

Result

안정적인 회전과 속도 구현



04 프로젝트 수행 경과

Key Issue 7 - (트롤러) 도킹 ArUco 문제

마커는 잘 보이는 시각 구간에서 자세를 확정하고, 이후에는 저장된 실제 Pose/Yaw 기준으로 직진 삽입

초기 문제

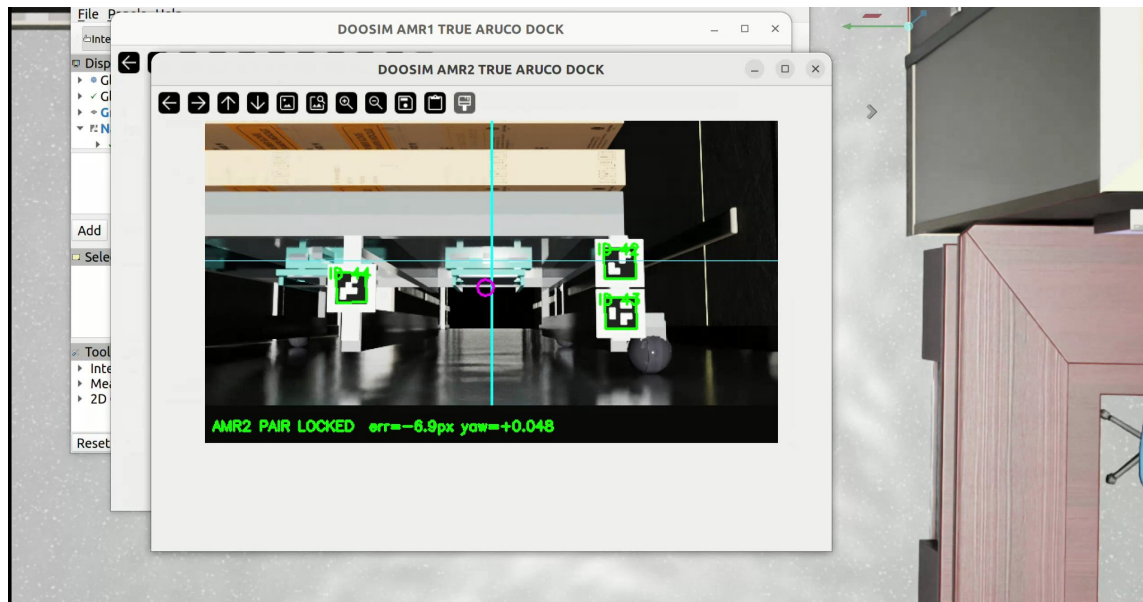
트레이에 가까워질수록 마커가 카메라 밖으로 사라짐
 → PAIR LOST 반복
 → 도킹 직전 정지

원인 판단

카메라가 트레이 아래로 들어가면 마커가 가려지는 것은 정상
 → 끝까지 추적하는 요구 자체가 잘못됨

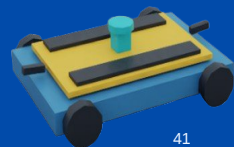
최종 해결

ONE-SHOT FULL PAIR LOCK
 → world pose / yaw 저장
 → dual direct insert



FULL PAIR LOCK 조건 | AMR1: 40/41+44 · AMR2: 44+42/43 · 중심 오차 $\pm 18\text{px}$ · 2 frames 안정화

05 자체 평가 의견



05 자체 평가 의견

프로젝트 결과물에 대한 완성도 평가

9.1/10

05 자체 평가 의견

개별 완성도 평가

평가 항목	점수	평가 근거
전체 시나리오 구현	10/10	OCR·결합·엘리베이터·MRI·복귀 흐름
자율주행 안정성	8/10	Nav2 경로 추종과 벽 충돌 방지
침대 결합 안정성	9/10	리프트와 Fixed Joint 반복 성공
비전/OCR 성능	9/10	환자 이름표 검증과 X축 정렬
시스템 통합 완성도	9/10	GUI/DB/ROS2/엘리베이터 연결
실무 활용 가능성	10/10	디지털 트윈 기반 사전 검증 가치

05 자체 평가 의견

주요 개선점, 보완할 점

1 설계 단계별 세부 구체화 (ASSET 간의 COLIDER)

ASSET 간의 COLIDER 충돌범위 확인 적용 TEST 우선적용
MAP도 포함 간단한 시뮬레이션 꼭 필수적으로 하기

2 외부 ASSET의 무한적인 신뢰는 하지않기

외형과 달리 동작하는 부분이나 적용되는 부분이 많이 달라
외부 ASSET의 활용을 이후에 생각하는게아닌 초반에 계획적으로
TEST할것

3 Action graph 및 기능을 더 적극활용하기

기본기능 활용 극대화 한 asset 제작하기

4 물리적 특성에 대한 설계를 잊지않기

물리적특성을 무시하고 생각대로 하다가 바퀴가 안굴러가는, 등
맵에서 바퀴가 춤춘다는,등 여라가지 상황때문에 작업시간이
증가함

05 자체 평가 의견

개인별 완성도 평가

김한준

- 어려웠던 점: 안정적인 자율주행 경로를 만드는 것이 어려웠다.
- 해결한 방법: 플래너를 보정하는 알고리즘을 만들어 부드럽게 주행하도록 만들어 해결했다.
- 새롭게 배운 기술: 플래너, 컨트롤러의 개념과 보정 방법

강대환

- 어려웠던 점: **action graph** 같은 **graph** 기능들의 활용이 어려웠다
- 해결한 방법: **python** 노드로 해결
- 새롭게 배운 기술: **isaac sim asset** 구성 및 **colider** 설정 다양한 **script python** 활용방법

서정민

- 어려웠던 점: **isaac sim** 물리적 특성과 두 **AMR**간 협동 동기 제어
- 해결한 방법: 물리 파라미터 최적화, 상태 공유 기반 협동 제어 및 우선순위 **Traffic control**
- 새롭게 배운 기술: **Isaac Sim, nav2 ndual AMR, aruco marker, PhsyX**

류호현

- 어려웠던 점: **GUI**상에 실제 **Map**상의 **AMR** 관련 실시간 위치를 표현하기 어려움
- 해결한 방법: **GUI** 상에 설정 좌표 오차 범위 내 **/world_pose**으로 판단
- 새롭게 배운 기술: **Flask, SQLite** 등 **Web** 관련 기술 활용 방법

조해벽

- 어려웠던 점: **isaacsim** 이 처음이보니 어떻게 사용해야 하는지 잘 몰랐다.
- 해결한 방법: **isaacsim** 강의 영상을 보고, 익숙해질때 까지 연습함
- 새롭게 배운 기술: **isaac sim** 설정, **nav2** 설정, **lidar** 사용법,

DOOSIM